

**AI-BASED TOXIC CONTENT AND POTENTIAL
CYBERBULLYING DETECTION:
MACHINE LEARNING BASED TEXT CLASSIFICATION
OF TWEETS**

A PROJECT REPORT

Submitted by

SARAH SIDDIQUI (25BCE10451)

MUDRIKA SHARMA (25BCE10452)

K KUSHAL VARMA (25BCE10447)

SUMEDHA KASHMIRA (25BCE10764)

NAVRUP KAUR (25BCE10350)

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in COMPUTER SCIENCE AND ENGINEERING (CSE-Core)

**SCHOOL OF COMPUTING SCIENCE AND ENGINEERING
VIT BHOPAL UNIVERSITY
KOTRIKALAN, SEHORE
MADHYA PRADESH - 466114**

OCTOBER 2026

BONAFIDE CERTIFICATE

VIT BHOPAL UNIVERSITY, KOTHRIKALAN, SEHORE
MADHYA PRADESH - 466114

Certified that this project report titled "**AI-Based Toxic Content and Potential Cyberbullying Detection: Machine Learning Based Text Classification of Tweets**" is the bonafide work of "**Sarah Siddiqui (25BCE10451), Mudrika Sharma (25BCE10452), K Kushal Varma (25BCE10447), Sumedha Kashmira (25BCE10764) and Navrup Kaur (25BCE10350)**" who carried out the project work under my supervision. Certified further that to the best of my knowledge the work reported at this time does not form part of any other project/research work based on which a degree or award was conferred on an earlier occasion on this or any other candidate.

The project is based on the research paper: K. M. O. Nahar, M. Alauthman, S. Yonbawi and A. Almomani, "Cyberbullying Detection and Recognition with Type Determination Based on Machine Learning," *Computers, Materials & Continua*, vol. 75, no. 3, pp. 5307-5319, 2023.

PROGRAM CHAIR

Dr. Vikas Panthi, Program Chair, CSE-
Core
School of Computing Science and
Engineering
VIT BHOPAL UNIVERSITY

PROJECT GUIDE

Dr. Vikas Panthi, Program Chair, CSE-
Core
School of Computing Science and
Engineering
VIT BHOPAL UNIVERSITY

The Project Exhibition I Examination is held on _____

ACKNOWLEDGEMENT

We take this opportunity to express our sincere gratitude to the School of Computing Science and Engineering, VIT Bhopal University, for providing the environment in which this project could be carried out from problem selection through to the final review. We wish to record our heartfelt thanks to our project guide, **Dr. Vikas Panthi**, Program Chair, CSE-Core, for continually guiding and actively participating in our project, for reviewing our model comparison plan when we were inclined to report accuracy alone, and for his valuable suggestions in completing the project work. His insistence that the combined models be measured on cost as well as on accuracy shaped the direction of this report. We would like to thank all the technical and teaching staff of the School of Computing Science and Engineering, who extended directly or indirectly all support in making the Google Colab environment, the dataset and the evaluation tooling available to us, and who helped us resolve the software dependency and storage issues encountered while running the pipeline. We are also grateful to the authors of the base research paper and of the related works surveyed in Chapter 2, whose published methods and results gave this project both its starting point and the benchmark against which our models were measured. Last but not least, we are deeply indebted to our parents and our teammates, who have been the greatest support while we worked day and night on the dataset, the code and the presentation to make this project a success.

THE AUTHORS

Sarah Siddiqui · Mudrika Sharma · K Kushal Varma · Sumedha Kashmira · Navrup Kaur

LIST OF SYMBOLS, NOMENCLATURE AND ABBREVIATIONS

Symbols and notation

Symbol	Meaning as used in this report	Defined in
n	Number of documents in the evaluation (test) set; here $n = 9,589$	Section 5.5.2; App. B.1
P	Number of toxic-class documents in the test set, not published and solved back from the reported metrics as ≈ 673	Section 5.5.2; App. B.1
TP, FP, FN, TN	True positives, false positives, false negatives and true negatives of the toxic class over those n tweets	Section 5.5.1; App. B.1
Acc, Prec, Rec	Accuracy $(TP + TN) / n$, precision $TP / (TP + FP)$ and recall $TP / (TP + FN)$, the last two taken on the toxic class	Sections 5.5.1 and 5.5.4; App. B.1
F1	Positive-class F-measure, $2 \cdot \text{Prec} \cdot \text{Rec} / (\text{Prec} + \text{Rec})$; the ensemble values printed by the review are weighted averages (Section 5.5.4)	Section 5.5.4; App. B.2
V	Size of the TF-IDF vocabulary, i.e. the number of features; $V = 26,409$, of which both reported parameter counts are exact multiples	Section 5.5.5; App. B.4
$w, 1 - w$	Fusion weights of the two models in a combined predictor, searched over 0.0 to 1.0 in steps of 0.1	Sections 4.2.4 and 5.2.3
a, l, f	The labels one tweet receives from the SVM path and from the Logistic Regression path of Strategy I, each in $\{0, 1\}$, and the arbitration rule f , which returns a when they agree and l otherwise	Section 5.5.3; App. B.3
0, 1	Class labels in the source corpus, 1 = Hatred and 0 = Non-Hatred; the deployed functions print words, not codes	Sections 3.3.1 and 4.5

Abbreviations

Item	Full form	Item	Full form
Colab	Google Colaboratory, the hosted notebook used	ML	Machine Learning
CPU	Central Processing Unit	NB	Naive Bayes
CSV	Comma-Separated Values	NLP	Natural Language Processing
CSE	Computer Science and Engineering	NLTK	Natural Language Toolkit for Python
F1	F-measure, the harmonic mean of precision and recall	RF	Random Forest
FN, FP, TP, TN	False negative, false positive; true positive, true negative	SVM	Support Vector Machine
GPU	Graphics Processing Unit, not used by this pipeline	TF-IDF	Term Frequency – Inverse Document Frequency
LDA	Latent Dirichlet Allocation, the base paper's topic model	URL	Uniform Resource Locator, stripped during cleaning
MB	Megabyte, read as 10^6 bytes	VIT	Vellore Institute of Technology

Named artefacts in the code

Identifier	What it is, and why it matters to the result	Described in
<code>train.csv</code>	The single notebook input: 31,962 tweets with their text and label column	Section 3.3.1; App. A
<code>clean_text()</code>	The cleaning function shared by the training and the serving path, so a live tweet is transformed exactly as a training tweet was	Sections 4.2.1 and 5.2.1
<code>TfidfVectorizer</code>	The feature extractor, fitted on the 22,373 training tweets only and reused unchanged on the test set	Sections 4.2.2 and 5.2.1
<code>LogisticRegression</code>	With <code>max_iter=1000</code> so the solver converges on a 26k-feature problem; the arbiter of Strategy I and the second member of both fusions	Sections 4.2.3 and 5.2.2
<code>LinearSVC()</code>	The most accurate single model, and the source of the signed margin that SVM + LR normalises before fusing	Sections 4.2.3 and 4.2.4
<code>MultinomialNB()</code>	Combined with LR by posterior averaging; its precision, recall and F1 were computed but not printed in the review	Sections 4.2.3 and 5.5.7
<code>RandomForestClassifier</code>	Trained on the identical matrix with <code>n_estimators=100</code> , <code>random_state=42</code> , <code>n_jobs=-1</code> ; no metric for it appears anywhere in the review	Sections 4.2.3 and 5.5.7
<code>predict_proba()[:, 1]</code> <code>decision_function()</code>	The two score streams the fusions consume: a posterior probability for NB + LR, a min-max normalised margin for SVM + LR	Sections 4.2.4 and 5.2.3
<code>np.arange(0.0, 1.1, 0.1)</code>	The eleven candidate fusion weights, each scored on the test set itself and the best accuracy kept	Sections 4.2.4 and 5.2.3
<code>average='weighted'</code>	The averaging mode of the ensemble metric calls, and the reason their F1 is not comparable with the single models'	Sections 5.2.3 and 5.5.4
<code>predict_svm_lr(text)</code> <code>predict_nb_lr(text)</code>	The two console entry points: one string in, one readable class out, the fused pair named in the identifier	Sections 4.5 and 5.2.4

Conventions followed in this report

"the review"	The twenty-three slide project review restated here; App. C maps each slide to a section	Derived values	Computed from reported ones (673 positives, 92.98% baseline, error counts) and proved in App. B
Reported values	Quoted as printed, with no re-rounding, so each can be checked against its slide	Base-paper values	Cited to Nahar et al. (2023) even where the review also prints them
Cross-references	Section, Table, Figure, Listing and Appendix numbers follow this report's own numbering, never slide numbers	Silences	Where the review printed no metric or no configuration, the report says so instead of filling the gap

Requirement identifiers are those of Chapter 3: FR-1 to FR-10 (Table 3.3), PR-1 to PR-5 and SR-1 to SR-4 (Table 3.4). Each symbol above keeps one meaning throughout the report.

LIST OF FIGURES AND GRAPHS

Figure No.	Title	Page No.
Figure 4.1	Overall system architecture, from the labelled corpus to the live label. Stages are coloured by function: ingestion, representation, modelling and evaluation, combination, serving.	19
Figure 5.1	Accuracy against toxic-class recall. Logistic Regression and the balanced SVM lie within 0.13 points of one another on accuracy while differing by 41.75 points on recall; accuracy cannot rank them, recall can.	28
Figure 5.2	All four reported models clear 93.9% accuracy, yet their recall spans 28.4% to 70.1%. The dashed line is the 92.98% majority-class reference derived in Section 5.5.2.	32
Figure 5.3	Each cost metric expressed as each model's share of the two-model total. SVM + LR occupies 25% of the combined prediction latency, 40% of parameters, 28% of size and 35% of non-zero values, while requiring 54% of the combined training time, the only line on which it is the more expensive choice.	33
Figure 5.4	Left: of the 673 toxic tweets, Logistic Regression flags 191 and misses 482, Linear SVM flags 381 and misses 292, the balanced SVM flags 472 and misses 201. Right: what that sensitivity costs on the 8,916 clean tweets, 19, 57 and 287 false alarms respectively.	34

LIST OF TABLES

Table No.	Title	Page No.
Table 1.1	Objectives of the project and the section in which each is addressed	3
Table 2.1	Comparative survey of the related work	7
Table 3.1	Hardware and software requirements	10
Table 3.2	Data requirement and partitioning	11
Table 3.3	Functional requirements	12
Table 3.4	Performance and security requirements	12
Table 4.1	Preprocessing operations applied to every tweet	15
Table 4.2	Base classifier configurations as implemented	17
Table 4.3	The two combination strategies as implemented	18
Table 4.4	Services exposed by each module	19
Table 5.1	Demonstrated input and output sessions	25
Table 5.2	Class balance implied by the published metrics, and the majority-class reference point	27
Table 5.3	Audit of the reported parameter counts against the vocabulary size	29
Table 5.4	Individual model results on the 9,589-tweet test set	31
Table 5.5	This project against the base paper on the same corpus	32
Table 5.6	Technical comparison of the two combined models	33
Table 6.1	Objectives of Chapter 1 and the outcomes achieved	36
Table 6.2	Applicability of the trained predictor by deployment context	37
Table 6.3	Limitations of the present work	38
Table 7.1	Prioritised recommendations	41
Table C.1	Mapping of the twenty-three review slides to this report	49

ABSTRACT

PURPOSE. Social media produce hateful and abusive text far faster than any human team can read, and the accuracy figures with which detectors are reported conceal how much of that content they actually catch. This project builds and evaluates a text-classification pipeline that labels a tweet as Hatred / Toxic or Non-Hatred, reproducing the machine-learning approach of Nahar et al. (2023), and extends it with a comparison of two combined classifiers judged on what they cost to run and store, not only on how often they are right.

METHODOLOGY. The 31,962 labelled tweets were cleaned by removing usernames, URLs, punctuation and stopwords and by Porter stemming, represented as a 26,409-dimensional TF-IDF matrix fitted on the training portion only, and split 70/30 into 22,373 training and 9,589 test tweets. Logistic Regression, Linear SVM, Naive Bayes and Random Forest were trained on the identical feature matrix, so every difference is attributable to the classifier alone. SVM + LR and NB + LR were then built by two mechanisms, an agreement rule with Logistic Regression as arbiter and a weighted score fusion with a grid-searched weight, and compared on prediction time, learned parameters, model size, non-zero values and training time, not only on accuracy, precision, recall and F1.

FINDINGS. Linear SVM reached the highest accuracy, 96.36%, with recall 56.61%; Logistic Regression reached the highest precision, 90.95%, on recall of only 28.38%; a class-weighted Linear SVM bought the best recall, 70.13%, by giving accuracy back to 94.91%; Naive Bayes reached 93.98%; all four lie within 1.5 points of the published baseline on the same corpus. SVM + LR is the cheaper deployment, 0.0029 s per prediction, 52,818 parameters and 0.40 MB, against 0.0087 s, 79,227 parameters and 1.01 MB for NB + LR, which trains faster. Four results qualify this: the metrics imply about 673 toxic tweets in the test set, so a classifier that never flags anything would still score 92.98%; the agreement rule is identical to Logistic Regression alone and so cannot show a gain; the ensemble F1 values are weighted averages, not positive-class F1; and the parameter counts are exact multiples of the vocabulary, describing the base estimators rather than the fused models.

CONCLUSION. Recall, not accuracy, is the operative measure for this task; a single linear SVM over a modest fraction of the vocabulary is a defensible deployment, and combining models as implemented buys cost rather than capability. The recommendations are reporting changes first: publish the class counts and the majority-class baseline beside every accuracy, record the class-weighted SVM's configuration, report the confusion matrix and the split seed, and select the fusion weight on a validation split, not on the test set.

Keywords: toxic content detection · cyberbullying · text classification · TF-IDF · logistic regression · support vector machine · naive Bayes · model ensembles · class imbalance · model cost

TABLE OF CONTENTS

Chapter No.	Title	Page No.
	BONAFIDE CERTIFICATE	i
	ACKNOWLEDGEMENT	ii
	LIST OF SYMBOLS, NOMENCLATURE AND ABBREVIATIONS	iii
	ABSTRACT	vii
1	PROJECT DESCRIPTION AND OUTLINE	1
2	RELATED WORK INVESTIGATION	5
3	REQUIREMENT ARTIFACTS	10
4	DESIGN METHODOLOGY AND ITS NOVELTY	15
5	TECHNICAL IMPLEMENTATION AND ANALYSIS	22
6	PROJECT OUTCOME AND APPLICABILITY	36
7	CONCLUSIONS AND RECOMMENDATION	40
	REFERENCES	43
APP. A	COMPLETE PIPELINE CODE	44
APP. B	DERIVATIONS BEHIND THE ANALYSIS	47
APP. C	REVIEW ARTIFACT TRACEABILITY	49
APP. D	REPRODUCTION CHECKLIST	51
1.1	Introduction	1
1.2	Motivation for the work	1
1.3	The project and the techniques it uses	2
1.4	Problem statement	2
1.5	Objectives of the work	3
1.6	Scope and limitations of scope	3
1.7	Organisation of the report	4
1.8	Summary	4
2.1	Introduction	5
2.2	Core area of the project: toxicity classification in short text	5
2.3	Existing approaches and methods	6
2.4	Pros and cons of the stated approaches	7
2.5	Issues and observations from the investigation	8

Chapter No.	Title	Page No.
2.6	Summary	9
3.1	Introduction	10
3.2	Hardware and software requirements	10
3.3	Specific project requirements	11
3.4	Summary	14
4.1	Methodology and goal	15
4.2	Functional modules: design and analysis	15
4.3	Software architectural design	19
4.4	Subsystem services	19
4.5	User interface design	20
4.6	Novelty of the proposed design	21
4.7	Summary	21
5.1	Outline	22
5.2	Technical coding and code solutions	22
5.3	Working layout of forms	25
5.4	Prototype submission	26
5.5	Test and validation	26
5.6	Performance analysis	31
5.7	Summary	35
6.1	Outcomes against the stated objectives	36
6.2	Behaviour of the trained system	36
6.3	Applicability	37
6.4	Limitations	38
6.5	Ethical and societal considerations	39
6.6	Summary	39
7.1	Conclusions	40
7.2	Recommendations	41
7.3	Future enhancements	42
7.4	Summary	42
2.3.1	Reynolds et al. (2011) - language-based features	6
2.3.2	Dinakar et al. (2011) - topic-specific classifiers	6
2.3.3	Waseem and Hovy (2016) - character n-grams	6
2.3.4	Davidson et al. (2017) - the offensive/hateful boundary	6
2.3.5	Nahar et al. (2023) - the base paper	7
2.5.1	The gap this project occupies	8
2.5.2	Imbalance must be reported, not assumed away	8

Chapter No.	Title	Page No.
2.5.3	Feature choice trades semantic precision for compactness	8
2.5.4	The omitted half of the base paper	9
3.3.1	Data requirement	11
3.3.2	Function requirements	12
3.3.3	Performance and security requirements	12
3.3.4	Look and feel requirements	13
3.3.5	Documentation and presentation requirements	14
4.2.1	Preprocessing module	15
4.2.2	Featurisation module	16
4.2.3	Base classifier module	17
4.2.4	Combination module	17
5.2.1	Preprocessing and featurisation	22
5.2.2	Training and evaluation	23
5.2.3	Ensemble construction	23
5.2.4	Serving functions	24
5.3.1	Demonstrated sessions	25
5.3.2	Test cases the project should add	25
5.5.1	Validation protocol	26
5.5.2	The omitted class balance, recovered, and what it does to accuracy	26
5.5.3	The agreement rule is equivalent to a single classifier	28
5.5.4	Metric consistency, and a scale mismatch between two tables	29
5.5.5	Parameter-count and model-size audit	29
5.5.6	Ambiguity in the latency measurement	30
5.5.7	Reproducibility conditions and results absent from the review	30
5.6.1	Individual classifiers	31
5.6.2	Comparison against the published baseline	32
5.6.3	Combined models on cost	33
5.6.4	Error structure and the choice of operating point	34
5.6.5	Interpretation	34
Table1.1	Objectives of the project and the section in which each is addressed	3
Table2.1	Comparative survey of the related work	7
Table3.1	Hardware and software requirements	10
Table3.2	Data requirement and partitioning	11

Chapter No.	Title	Page No.
Table3.3	Functional requirements	12
Table3.4	Performance and security requirements	12
Table4.1	Preprocessing operations applied to every tweet	15
Table4.2	Base classifier configurations as implemented	17
Table4.3	The two combination strategies as implemented	18
Table4.4	Services exposed by each module	19
Table5.1	Demonstrated input and output sessions	25
Table5.2	Class balance implied by the published metrics, and the majority-class reference point	27
Table5.3	Audit of the reported parameter counts against the vocabulary size	29
Table5.4	Individual model results on the 9,589-tweet test set	31
Table5.5	This project against the base paper on the same corpus	32
Table5.6	Technical comparison of the two combined models	33
Table6.1	Objectives of Chapter 1 and the outcomes achieved	36
Table6.2	Applicability of the trained predictor by deployment context	37
Table6.3	Limitations of the present work	38
Table7.1	Prioritised recommendations	41
TableC.1	Mapping of the twenty-three review slides to this report	49
Figure4.1	Overall system architecture, from the labelled corpus to the live label. Stages are coloured by function: ingestion, representation, modelling and evaluation, combination, serving.	19
Figure5.1	Accuracy against toxic-class recall. Logistic Regression and the balanced SVM lie within 0.13 points of one another on accuracy while differing by 41.75 points on recall; accuracy cannot rank them, recall can.	28
Figure5.2	All four reported models clear 93.9% accuracy, yet their recall spans 28.4% to 70.1%. The dashed line is the 92.98% majority-class reference derived in Section 5.5.2.	32
Figure5.3	Each cost metric expressed as each model's share of the two-model total. SVM + LR occupies 25% of the combined prediction latency, 40% of parameters, 28% of size and 35% of non-zero values, while requiring 54% of the combined training time, the only line on which it is the more expensive choice.	33

Chapter No.	Title	Page No.
Figure5.4	Left: of the 673 toxic tweets, Logistic Regression flags 191 and misses 482, Linear SVM flags 381 and misses 292, the balanced SVM flags 472 and misses 201. Right: what that sensitivity costs on the 8,916 clean tweets, 19, 57 and 287 false alarms respectively.	34
Listing5.1	Text cleaning and vectorisation	22
Listing5.2	Fitting the four base classifiers and scoring them	23
Listing5.3	Strategy I (agreement rule) and Strategy II (weighted score fusion)	23
Listing5.4	The two console-facing inference functions	24
ListingA.1	Complete pipeline, from CSV to a live label	44

CHAPTER - 1

PROJECT DESCRIPTION AND OUTLINE

1.1 Introduction

Platforms such as Twitter produce text continuously and at a volume no human team can read in full. A fraction of that text is hateful, toxic or abusive, and exposure to it is associated with measurable harm to mental health, which makes early detection a public-interest problem rather than a convenience feature. Manual review of millions of posts is slow and inconsistent, so the practical alternative is an automated classifier that screens content first and routes only the flagged subset to a human moderator.

The academic form of this problem is *text classification*: given a labelled corpus of messages, learn a mapping from message text to a toxicity label that generalises to messages never seen in training. This project adopts that framing. It reproduces the pipeline structure of the base research paper, cleaning, term-weighting, supervised classification and evaluation, and adds a comparison that the base paper does not make: what happens when two classifiers are asked to decide together instead of one being asked alone, and what that combination costs to run.

The work was executed in Python on Google Colab using the scikit-learn, pandas, NumPy, NLTK and Matplotlib libraries, and was presented for review as a twenty-three slide deck. This report restates that review in written form and, in Chapter 5, subjects its numbers to a consistency audit that the slides themselves do not attempt.

1.2 Motivation for the work

Three motivations shaped the choice of problem.

Harm at scale. Cyberbullying and hate speech are not edge cases of platform life; they are a persistent cost borne disproportionately by younger users and by targeted communities. A detector that flags even a fraction of abusive posts before a human sees them changes the working conditions of moderators and the exposure of victims.

The limits of manual review. Volume alone rules out exhaustive human screening, and consistency problems remain even where headcount exists. An automated first pass is the only mechanism that scales, and its value depends on being cheap enough to run on every incoming post.

The accuracy illusion. Published results in this area, including those of the base paper, are reported primarily as accuracy. On a corpus in which the abusive class is a small minority, accuracy is the least informative available measure: a classifier that never flags anything looks

excellent. The project was therefore motivated by a second question as much as the first, namely which metric actually separates these models, and by the observation that a *cost* comparison between combined models is closer to what an engineering team needs than another decimal place of accuracy.

1.3 The project and the techniques it uses

The system is a supervised binary text classifier. The techniques it chains together are standard in natural language processing, and each is present for a reason.

Text preprocessing. Raw tweets contain mentions, links, punctuation and case variation that carry no signal for toxicity and that inflate the feature space. Cleaning removes them, stopword removal discards high-frequency function words, and Porter stemming merges morphological variants so that "hating", "hates" and "hated" occupy one feature.

TF-IDF featurisation. A classifier needs numbers, not words. Term Frequency - Inverse Document Frequency weights each surviving stem by how often it appears in the tweet and how rare it is across the corpus, so that a distinctive term contributes more than a common one. The result for this project is a 26,409-column matrix that is 99.97% sparse, which is precisely the shape of data linear text classifiers handle well.

Linear classifiers. Logistic Regression and Linear SVM find a hyperplane in that 26,409-dimensional space; the former estimates probabilities, the latter maximises a margin. Naive Bayes applies Bayes' rule with a feature-independence assumption, which is violated by real language yet works well on counts. Random Forest averages over decision trees, offering a non-linear contrast to the other three.

Model combination. The final stage pairs classifiers so that two views contribute to one decision. Two mechanisms are implemented and reported in Chapter 4: an agreement rule with a fixed arbiter, and a weighted fusion of decision scores whose mixing weight is searched on a grid.

Evaluation. Accuracy, precision, recall and F1 are computed on the unseen test split, alongside confusion matrices; the combined models are additionally compared on prediction time, learned parameters, model size, non-zero learned values and training time.

1.4 Problem statement

Given a corpus of labelled tweets, build a reproducible machine-learning pipeline that classifies an arbitrary input text as **Hatred / Toxic** or **Non-Hatred**, and determine whether combining two classifiers yields a predictor preferable to either classifier alone, measured not only on accuracy but on the cost of running and storing the resulting model.

Three constraints follow from that statement and shape every design decision in Chapter 4. The features must be shared across all candidate models, so that a comparison of classifiers is not confounded by a comparison of representations. The test split must remain unseen during training, so that reported scores reflect generalisation. And the final artefact must respond to a freshly typed sentence, since a moderation tool that cannot classify a single post in real time cannot sit in front of a live stream.

1.5 Objectives of the work

Table 1.1 — Objectives of the project and the section in which each is addressed

#	Objective	Addressed in	Outcome
1	Follow the general machine-learning approach described in the base research paper	Sections 3.3.1, 4.1, 4.2, 5.6	Achieved - scores land within 1.5 points of the published baseline
2	Compare individual models and develop two combined models	Sections 4.2.3, 4.2.4, 5.6	Achieved; the combination rule is qualified in Section 5.5.3
3	Build the final combined models SVM + Logistic Regression and Naive Bayes + Logistic Regression	Section 5.2.4, Table 5.3	Achieved
4	Use more than one classifier in the final prediction approach	Sections 4.2.4, 4.6	Achieved through two distinct mechanisms
5	Compare the combined models on technical measures, not accuracy alone	Section 5.6.3, Figure 5.4	Achieved - five cost dimensions reported
6	Support real-time usage on arbitrary text	Sections 4.5, 5.3	Achieved - single-text inference demonstrated

1.6 Scope and limitations of scope

The deliverable is a working classification pipeline in Python together with a measured comparison of four individual classifiers and two combined predictors, and interactive functions that label arbitrary text.

The project is explicitly a *binary* toxicity detector. It does not classify the *type* of bullying, which is the second half of the base paper's contribution and is discussed at Section 2.5.4. It does not detect bullying by user, by thread or by image. Deployment is out of scope: no service, user interface or moderation queue was built, and the "real-time" contribution is a callable Python function rather than a running system. English-language tweets are the only input distribution; no multilingual or code-mixed support was attempted.

1.7 Organisation of the report

The remainder of this report is organised as follows. **Chapter 2** investigates related work, summarising five published approaches, their advantages and their limitations, and deriving from them the gap this project occupies. **Chapter 3** records the requirement artifacts: the hardware and software environment, and the data, functional, performance, security and look-and-feel requirements. **Chapter 4** presents the design methodology and its novelty, covering the pipeline modules, the software architecture, the subsystem services, the user interface and the two combination mechanisms. **Chapter 5** documents the technical implementation and analysis: the code, the working layout of the interactive forms, the prototype, the test and validation procedure including a consistency audit of the reported metrics, and the performance analysis with graphs. **Chapter 6** states the project outcome and its applicability to moderation settings, with the limitations and ethical considerations. **Chapter 7** draws the conclusions and makes prioritised recommendations. **References** and four **Appendices** follow, the last of which lists the steps needed to reproduce every number quoted here.

1.8 Summary

Chapter 1 defined the problem, the motivation and the six objectives, listed the techniques the project chains together, and bounded the scope to binary detection of hateful and toxic text in English tweets with a real-time callable as the interface. The next chapter examines what previous researchers have achieved on this problem and what remains unaddressed.

CHAPTER - 2

RELATED WORK INVESTIGATION

2.1 Introduction

This chapter reviews the published work on detecting cyberbullying, hate speech and toxic language in short social-media text, with the aim of establishing three things: what performance has already been claimed on this problem, which techniques produced it, and what those studies leave unexamined. The review is organised around the five studies recorded in the project's literature table, of which one, Nahar et al. (2023), is the base paper this project reproduces and extends.

2.2 Core area of the project: toxicity classification in short text

All five studies sit in the same applied area, and the area has four properties that explain both the reported numbers and their limitations.

Short, noisy documents. A tweet supplies a handful of words, heavily informal, with non-standard spelling, emphasis repetition, emoji and embedded handles. Feature spaces built from such text are large and extremely sparse, which is why bag-of-words and n-gram representations with TF-IDF weighting dominate this literature rather than syntactic parsers.

Subjective labels. "Hateful" is a judgement, not a measurement. Crowdsourced annotation protocols require several coders per item precisely because individuals disagree, and disagreements concentrate in the region that matters most: content that is offensive without being hateful.

Severe class imbalance. Corpora assembled by querying for abusive vocabulary contain a majority of non-hateful material, and the explicitly hateful subset is typically under a tenth of the data. Accuracy on such a corpus is therefore weakly related to practical usefulness, a point this project tests directly in Section 5.5.2.

Concept drift. Abusive language changes as terms are flagged and replaced, so a model is valid for the period its data covers. None of the studies reviewed here re-trains or re-validates over time.

2.3 Existing approaches and methods

2.3.1 Reynolds et al. (2011) - language-based features

Reynolds and colleagues were among the first to treat cyberbullying detection as a supervised text classification problem, extracting language-based features from message text and training machine-learning classifiers on them. Their contribution to this project is the establishing result that cyberbullying can be detected automatically from text alone, without metadata. Their method's weakness is the same as that of any lexicon-anchored approach: it relies on a limited set of offensive words, so it detects the *presence of vocabulary associated with abuse* rather than abusive *use* of language.

2.3.2 Dinakar et al. (2011) - topic-specific classifiers

Dinakar and colleagues built separate classifiers for distinct bullying topics such as appearance and intelligence, rather than one general model. Their finding, that topic-specific classifiers outperformed a single general classifier, is the clearest published evidence in this list that toxicity is not one phenomenon. The cost is data: a labelled dataset is needed for each topic, which makes the approach expensive to extend and explains why this project, with one binary label column, does not attempt it.

2.3.3 Waseem and Hovy (2016) - character n-grams

Waseem and Hovy examined which *features* actually predict hate speech on Twitter, comparing word-level and character-level representations and reporting that character n-grams with Logistic Regression gave strong results. Their result matters for noisy text because character n-grams do not require a word to be in a vocabulary: a deliberately misspelled slur still shares substrings with its canonical form. The limitations they document, subjective annotation and a comparatively small dataset, are limitations of the field rather than of their method.

2.3.4 Davidson et al. (2017) - the offensive/hateful boundary

Davidson and colleagues studied automated hate-speech detection and named the problem that most threatens the validity of every result in this area: hate speech is hard to tell apart from language that is merely offensive. Using TF-IDF with Logistic Regression and SVM, they showed that models trained on hate labels systematically misclassify offensive-but-not-hateful tweets. Because this project's corpus is the hatred/non-hatred dataset used by the base paper and is derived from the same Twitter collection, this finding applies to our results without qualification, and is the reason Section 5.3.2 proposes a specific test case for it.

2.3.5 Nahar et al. (2023) - the base paper

The base paper proposed a supervised methodology for detecting bullying, harassment and hate-related text using SVM, Naive Bayes, Logistic Regression and Random Forest, and additionally applied Latent Dirichlet Allocation as an unsupervised step to estimate the *type* of the offending text, reporting three resulting topic groups with token shares of 39.3% sexist, 30.4% religious and 30.3% other. Their corpus was the Twitter sentiment-analysis dataset distributed through Analytics Vidhya, described as 32k tweets labelled hatred or non-hatred, split 70% training against 30% testing. Reported accuracies were 95% for Logistic Regression, 94.97% for SVM, 94.66% for Random Forest and 93.1% for Naive Bayes.

Two properties of that work define this project's starting point. Its classifiers are evaluated *individually*, each on its own merits, and accuracy is the headline measure in every reported comparison.

2.4 Pros and cons of the stated approaches

Table 2.1 — Comparative survey of the related work

Study	Method	Reported strength	Documented weakness
Nahar et al. (2023)	TF-IDF with SVM, LR, NB, Random Forest; LDA for type	LR 95%, SVM 94.97%, RF 94.66%, NB 93.1% accuracy	Models used individually, never combined; accuracy is the main metric
Reynolds et al. (2011)	Language-based features with ML classifiers	Showed cyberbullying can be detected automatically from text	Relies on a limited set of offensive words
Dinakar et al. (2011)	Separate classifiers per topic	Topic-specific classifiers beat one general classifier	Requires a labelled dataset for every topic
Waseem and Hovy (2016)	Character n-grams with Logistic Regression	Character n-grams strong for hate-speech detection	Annotation subjective; dataset small
Davidson et al. (2017)	TF-IDF with LR and SVM	Identified the hate/offensive boundary as the core difficulty	Misclassifies offensive-but-not-hateful tweets

Read across the row, the trajectory is instructive. Reynolds et al. established that lexical features suffice. Dinakar et al. showed that splitting the problem by context outperforms one general model, at the price of needing labels per context. Waseem and Hovy moved from words to character n-grams, tolerating the informal spelling that dominates social media. Davidson et al. identified the validity threat that undermines all of them, that "offensive" and "hateful" are not separable by lexicon. Nahar et al. combined the mature recipe, TF-IDF with

four standard classifiers on a large labelled corpus, and reported a tight band of accuracies between 93.1% and 95%.

What none of the five does is treat the *combination* of two classifiers as an object of study, and what none reports alongside accuracy is the cost of the resulting model in memory and latency, which is the quantity that decides whether a detector can be run on every post.

2.5 Issues and observations from the investigation

2.5.1 The gap this project occupies

The base paper's own limitations, models evaluated individually and accuracy as the primary metric, are the gap this project targets. Its response is twofold: it combines classifiers into SVM + LR and NB + LR so that the combination becomes the unit of study, and it evaluates those combinations on prediction time, learned parameters, model size, non-zero learned values and training time in addition to accuracy, using the same TF-IDF features for both so that the comparison is fair. This is a modest and well-chosen gap: measuring the cost of an ensemble is closer to what an engineering team actually needs when it decides what to run per request than another accuracy figure would be.

2.5.2 Imbalance must be reported, not assumed away

The investigation surfaced a methodological issue that applies to this project as much as to the studies reviewed. Accuracy on an imbalanced corpus can be high while the model fails at the task. Section 5.5.2 derives from this project's own published metrics that the test set holds about 673 toxic tweets in 9,589, roughly 7%, so a classifier that always answered "Non-Hatred" would score 92.98% accuracy. Two of the four models in Table 5.1 sit within two points of that trivial reference. Any conclusion drawn from this literature, including our own, is unsafe unless the class balance is stated next to the accuracy.

2.5.3 Feature choice trades semantic precision for compactness

Waseem and Hovy's character n-gram result is a corrective to the cleaning regime this project inherits. Removing stopwords discards function words that carry negation, and "not" is among the most consequential tokens in this domain; once removed, "not acceptable" and "acceptable" become indistinguishable to the vectorizer. Aggressive stemming merges words whose distinction a moderator would care about. Neither choice is wrong, and both are standard, but they are choices with a measurable price that the base paper and this project alike do not quantify.

2.5.4 The omitted half of the base paper

The base paper's title ends in "with Type Determination", and its second contribution is the LDA topic estimation that classifies bullying into sexist, religious and other types. This project reproduces the detection half and omits the type-determination half, a legitimate scoping decision for a review submission and consistent with its own stated objective of following the *general* approach of the paper. The consequence is worth stating plainly: a binary Hatred/Non-Hatred output tells a moderator *that* a post is toxic but not *what kind* of toxicity it is, and real moderation routing depends on the second half.

2.6 Summary

Five studies were examined, from the first demonstration that text alone can signal cyberbullying to the identification of the hate/offensive boundary as the central validity threat. The base paper supplies the corpus, the preprocessing regime, the four classifiers and the accuracy figures against which this project is measured. The investigation locates the project's contribution in the comparison of *combined* models on cost as well as accuracy, and flags two conditions on which any such comparison depends, that class imbalance be reported alongside accuracy, and that the combination rule be one capable of differing from its own components. Chapter 3 turns from the literature to the artifacts this project required.

CHAPTER - 3

REQUIREMENT ARTIFACTS

3.1 Introduction

This chapter records the artifacts the project required and the requirements those artifacts impose. It covers the development environment, the data requirement and its partitioning, the functional requirements the pipeline must satisfy, the performance and security requirements that bound it, and the look-and-feel requirement that defines its user-visible behaviour. Requirements are stated so that each can be checked against the implementation in Chapter 5 and the outcomes in Chapter 6.

3.2 Hardware and software requirements

The project was built and run entirely inside a hosted notebook, so the hardware requirement is a browser and a connection rather than a machine specification.

Table 3.1 — Hardware and software requirements

Category	Requirement	Purpose in this project
Language	Python 3	Implementation of the whole pipeline
Platform	Google Colaboratory	Editing, execution and storage of the notebook
Core libraries	pandas, NumPy	Loading the CSV, column handling, score tables
NLP library	NLTK	English stopwords and the Porter stemmer
ML library	scikit-learn	TfidfVectorizer; LogisticRegression, LinearSVC, MultinomialNB, RandomForestClassifier; metric functions
Plotting	Matplotlib	Graphs used in Chapter 5
Dataset	Labelled tweet corpus in CSV form (<code>train.csv</code>), 31,962 rows	Training and evaluation
Browser	Google Chrome	Colab client
Hardware	Laptop or desktop with internet access	Access to the hosted notebook
Memory / storage	No specific requirement stated or encountered	Sparse matrices keep the footprint low
Accelerator	None required	All four models train on CPU in under one second each

The software stack is entirely CPU-scoped, and this is a design decision worth appreciating rather than a deficiency: a TF-IDF plus linear-model pipeline trains in under a second per model on 22,373 documents, so a deep-learning stack would have added cost without a documented accuracy need. The absence of a hardware requirement beyond a browser also means the project is reproducible on the free tier of the hosted notebook service, which matters for a student submission.

3.3 Specific project requirements

3.3.1 Data requirement

The single data requirement is a labelled corpus of tweets with a binary hatred indicator.

Table 3.2 — Data requirement and partitioning

Property	Value
Source file	<code>train.csv</code>
Labelled tweets	31,962
Label set	Hatred / Toxic (1), Non-Hatred (0)
Provenance	Analytics Vidhya Twitter sentiment-analysis corpus, 32k tweets labelled hatred or non-hatred, as used by the base paper
Training split	70% - 22,373 tweets
Test split	30% - 9,589 tweets
Feature dimensionality	26,409 TF-IDF features
Training matrix shape	22,373 x 26,409
Test matrix shape	9,589 x 26,409
Matrix sparsity	99.97%
Class balance	Not stated in the review presentation; derived as approximately 673 toxic tweets in the test set, about 7.0% (Section 5.5.2)

Three requirements attach to how this data is handled. The vectorizer must be fitted on the training portion only and merely applied to the test portion, so that no test vocabulary influences the feature space. The test split must not be used for training or for tuning. And the split arithmetic must be verifiable, which it is: $22,373 + 9,589 = 31,962$ exactly, and $9,589 / 31,962 = 30.00\%$, so the partition is an exact row count rather than an approximate one.

3.3.2 Function requirements

Table 3.3 — Functional requirements

ID	Requirement
FR-1	Load the labelled corpus from CSV and expose the tweet text with its label column
FR-2	Clean each tweet by lowercasing, removing mentions and URLs, removing punctuation and special characters, removing stopwords and applying Porter stemming
FR-3	Split the corpus 70/30 into training and test portions
FR-4	Fit TF-IDF on the training portion and transform both portions to the same 26,409-feature space
FR-5	Train Logistic Regression, Linear SVM, Naive Bayes and Random Forest on the identical feature matrix
FR-6	Predict labels for the unseen test tweets and compute accuracy, precision, recall, F1 and the confusion matrix for each model
FR-7	Combine SVM with Logistic Regression, and Naive Bayes with Logistic Regression, into two final predictors
FR-8	Compare the two combined predictors on prediction time, learned parameters, model size, non-zero learned values and training time
FR-9	Accept one arbitrary string at the console, apply the same cleaning and transformation, and return its class as a readable label
FR-10	Produce plots and a formatted results table suitable for a review presentation

3.3.3 Performance and security requirements

Table 3.4 — Performance and security requirements

ID	Requirement	Value or constraint	Status
PR-1	Single-text inference latency	About 0.003 s for SVM + LR	Met as reported; unit needs clarification (Section 5.5.6)
PR-2	Model footprint	0.40 MB (SVM + LR) / 1.01 MB (NB + LR)	Met - small enough to ship to a client
PR-3	Training time	0.9036 s (SVM + LR) / 0.7555 s (NB + LR)	Met - retraining is effectively free
PR-4	Memory during training	Sparse matrices; no dense 22,373 x 26,409 array	Met - runs in a hosted notebook
PR-5	Reproducibility	Same split and same settings reproduce the reported figures	Partly met - no seed documented for the split (Section 5.5.7)

ID	Requirement	Value or constraint	Status
SR-1	Data privacy	Corpus is public tweets; no personal data collected by the team	Met
SR-2	Exposure to harmful content during development	Team read toxic examples while building the pipeline	Mitigated by CSV-based review rather than platform browsing
SR-3	Label integrity	Labels are third-party human judgements, not ground truth	Accepted limitation (Section 6.5)
SR-4	Misuse as an authority	A 28-70% recall detector cannot be a sole decision-maker	Design constraint carried into Section 6.5
SR-5	Credential and secret handling	No secrets in the notebook; hosted environment only	Met

Performance requirement PR-2 is the decisive number for the deployment contexts named in Section 6.3: a model of 0.40 MB can be embedded in a per-request service or shipped to a client without an inference fleet, whereas the same classification from a hosted deep-learning endpoint would need server capacity per request.

3.3.4 Look and feel requirements

The interface requirement is deliberately minimal. The system presents itself as a console prompt that accepts free text and prints one line of output naming the combined model and its verdict, in the style `SVM + Logistic Regression: Non-Hateful`. Three constraints were imposed on that presentation.

Immediate. The response must appear as soon as the text is submitted, with no queueing and no batch step, because the demonstration case is a live moderation decision.

Self-describing. The output names which combined model produced the verdict, so that a reviewer can compare SVM + LR and NB + LR on the same input rather than inferring which was consulted.

Consistent vocabulary. The class strings must be identical everywhere the model speaks. Two spellings appear in the review material, "Hatred / Toxic" and "Hateful / Toxic" for class 1 with the matching negations, and Section 5.3 records where each occurs. A deployed system must emit one vocabulary, because downstream moderation actions are commonly keyed on the label string.

3.3.5 Documentation and presentation requirements

The review itself set three artifacts requirements: a runnable notebook, a slide deck presenting introduction, existing work, proposed work, methodology, novelty, real-time usage, requirements, architecture, literature review, module descriptions, workflow, implementation, demo, snapshots, testing, results and conclusion, and captured output snapshots evidencing that the code was executed rather than merely written. This report is the fourth artifact, and it adds the audit in Chapter 5 that a slide deck of that structure has no room for.

3.4 Summary

Chapter 3 recorded the environment, a single data requirement of 31,962 labelled tweets partitioned 70/30 into a 26,409-feature space, ten functional requirements, five performance and five security requirements each with a stated status, and the look-and-feel constraints on the console interface. With the requirements fixed, Chapter 4 presents the design that satisfies them.

CHAPTER - 4

DESIGN METHODOLOGY AND ITS NOVELTY

4.1 Methodology and goal

The goal of the design is a pipeline in which the only variable between compared models is the classifier itself. Every other choice, corpus, cleaning, feature space, split and evaluation protocol, is held fixed and shared. The methodology follows the general supervised approach of the base paper, extended at one point: after the individual classifiers are scored, the design asks what a *combination* of two of them costs as well as what it achieves.

The pipeline is a linear sequence with a single branch at the end: a shared representation feeds four independent classifiers, whose outputs are then re-paired into two combined predictors.

Preprocessing to remove text with no toxicity signal; **TF-IDF featurisation** to convert words into weighted counts; **model training** on the 22,373-tweet portion; **evaluation** on the 9,589-tweet portion held out from training; and **model combination** for the two final predictors, followed by the serving path that labels arbitrary text.

Two properties of this structure deserve emphasis before the modules are described. First, the dataset flows through one featurisation step, so both combined models use identical TF-IDF features, which is what allows the technical comparison in Chapter 5 to be read as a comparison of model structures rather than of representations. Second, the test set is fixed before any model sees it and is kept unseen during training, with predictions evaluated against the actual test labels.

4.2 Functional modules: design and analysis

4.2.1 Preprocessing module

Table 4.1 — Preprocessing operations applied to every tweet

Step	Operation	Purpose and design note
1	Lowercase the text	Collapse case variants before vocabulary construction
2	Remove @username mentions	Handles carry no toxicity signal and would fragment the vocabulary
3	Remove http... and www... URLs	Strips link noise; performed before punctuation removal so protocol characters do not leave fragments
4	Remove punctuation and special characters	Retain letters and whitespace only

Step	Operation	Purpose and design note
5	Collapse runs of whitespace and trim	Normalise spacing for tokenisation
6	Drop English stopwords (NLTK)	Volume reduction; see the negation caveat below
7	Porter-stem every remaining token	Merge morphological variants into one feature

The stated purpose is to reduce unnecessary text and prepare tweets for feature extraction, and the design achieves that. Two of the seven steps are, however, trade-offs rather than pure gains, and it is worth analysing them rather than inheriting them silently. Step 6 removes negation cues, so "not acceptable" and "acceptable" collapse to the same features; in a task where sarcasm and denial are common this can cost precision. Step 7 is aggressive by design, mapping "hating", "hates" and "hated" to one stem, which helps sparsity but can merge words a moderator would distinguish. Both steps match the cleaning logic of the base paper, so retaining them also preserves comparability with the published baseline - the reason the project keeps them and proposes an ablation rather than an unmeasured change (Section 7.2).

4.2.2 Featurisation module

The featuriser converts each cleaned tweet into a 26,409-dimensional TF-IDF vector, weighting a term by its frequency in the tweet against its rarity across the corpus. It is fitted on training data and then used to transform the test data, which is why both matrices carry exactly 26,409 columns.

Two analytical observations follow from the design. The vocabulary size is not a tuned hyperparameter but a consequence of the corpus and of the cleaning sequence, since it is the count of distinct surviving stems; changing step 6 or 7 changes it. And the reported sparsity of 99.97% is coherent with the pipeline, which is worth verifying because an implausible sparsity would indicate a broken featuriser: at 0.03% density, one row of 26,409 cells carries 7.92 non-zero entries, so the test matrix of 9,589 rows holds 75,971 non-zero values, and a short tweet retaining six to eight distinct stems after cleaning is exactly what one expects. The training split computes to the same 7.92 per document, as it must from one shared vocabulary.

4.2.3 Base classifier module

Table 4.2 — Base classifier configurations as implemented

Model	Class used	Configuration	Design rationale
Logistic Regression	LogisticRegression	max_iter=1000	Iteration ceiling raised so the solver converges on a 26k-feature problem
Linear SVM	LinearSVC	library defaults (C=1.0, loss='squared_hinge')	Linear kernel suits high-dimensional sparse input; margins are reused by the fusion design
Naive Bayes	MultinomialNB	library defaults (alpha=1.0 Laplace smoothing)	Built for count-like features; yields probabilities directly
Random Forest	RandomForestClassifier	n_estimators=100, random_state=42, n_jobs=-1	Non-linear contrast to the other three; deterministic seed
Balanced Linear SVM	Linear SVM variant	not documented in the review material	Appears in the results; the mechanism is unspecified (Section 5.5.7)

All models are compared on the same TF-IDF features. The explicit max_iter=1000 on logistic regression is worth noting because at its default ceiling that solver frequently fails to converge on a matrix of this width, reporting a warning while returning a worse model. random_state=42 on the forest is the only determinism control in the design, and Section 5.5.6 discusses what that does and does not make reproducible. On high-dimensional sparse text, linear models are the appropriate baseline: the feature space is large enough that kernel tricks rarely help, per-document vectors are so sparse that a linear separator is usually sufficient, and the training times in Table 5.3 confirm that the entire model fits in under a second.

4.2.4 Combination module

The design contains two combination mechanisms, and separating them is essential to understanding the results, because they have very different properties.

Table 4.3 — The two combination strategies as implemented

	Strategy	Decision rule	Where specified
I	Agreement with fixed arbiter	If the two classifiers agree, take that label; if they disagree, take the Logistic Regression label	Review listing for combined models; both serving functions
II	Weighted score fusion	Blend the two decision scores with weights (w , $1-w$), threshold the blend at 0.5; w selected by search over 0.0 to 1.0 in steps of 0.1	Review code snapshots

Strategy II is the more interesting design. Naive Bayes exposes calibrated-in-form probabilities through `predict_proba`, so for NB + LR the two probability streams are averaged directly. A linear SVM has no probability output, so for SVM + LR the raw `decision_function` margins are min-max normalised to the unit interval before blending, after which the fused score is compared with 0.5 and the weight that maximises test accuracy is retained together with its precision, recall and F1.

The min-max normalisation is a real engineering detail with a real consequence. It makes the two score streams commensurable, but it also ties the fusion weights to the score range of the data being evaluated, so the weights are selected on evaluation data. The practical implication is that the chosen w is a property of the test split rather than of the model, which is why Section 7.2 recommends moving the search to a validation split.

Strategy I has a structural defect that must be stated at design level rather than discovered in results: under that rule the combined predictor returns Logistic Regression's label for *every* input, whether the two models agree or not, so it cannot differ from LR. The proof and its consequences are given at Section 5.5.3. The design is nonetheless retained here because it is what the serving functions implement, and because it establishes the baseline against which Strategy II should be reported.

4.3 Software architectural design

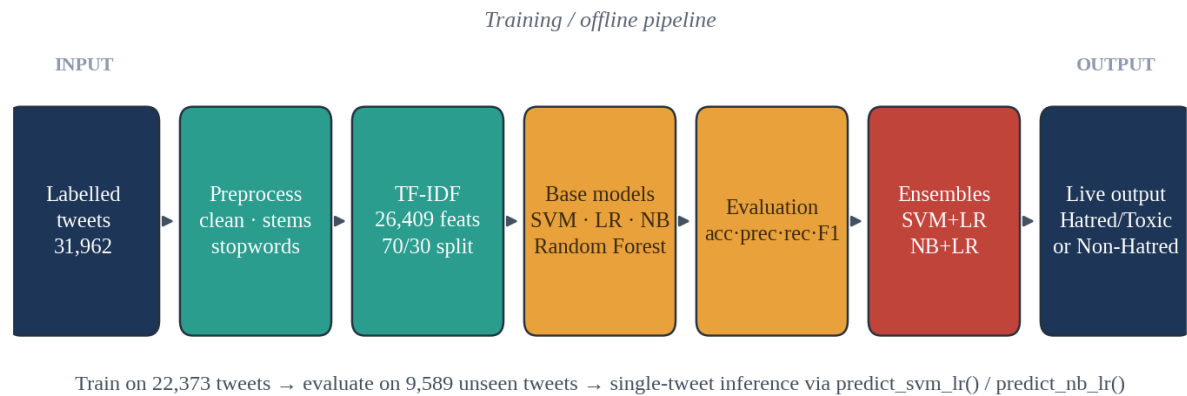


Figure 4.1 — Overall system architecture, from the labelled corpus to the live label. Stages are coloured by function: ingestion, representation, modelling and evaluation, combination, serving.

The architecture divides cleanly into an offline half and an online half. The offline half, the first five stages, is executed once per training run and produces the fitted vectorizer and four fitted estimators. The online half, the last stage, is executed per request: it reuses the *same* fitted vectorizer rather than relearning anything, which is what makes the serving path a single line of Python and what keeps latency in the millisecond range.

No component in the diagram requires a GPU, a serving framework or a model server, and no component communicates with another over a network; the entire system is in-process and single-threaded except for Random Forest's `n_jobs=-1`. That simplicity is a design property worth claiming, since it is what makes the 0.40 MB artifact in Table 5.3 deployable in places a deep-learning endpoint could not go.

4.4 Subsystem services

Table 4.4 — Services exposed by each module

Each module exposes one service to the rest of the system. Describing them as services makes the data flow explicit and shows what the serving path actually depends on.

Service	Input	Output	Consumed by
Cleaning	raw text	normalised, stemmed token string	featurisation, training, both serving functions
Featurisation	cleaned string + fitted vectorizer	26,409-column sparse row vector	all classifiers, both serving functions
Training	training matrix + labels	fitted estimator (x4)	evaluation, combination

Service	Input	Output	Consumed by
Evaluation	predicted labels + true test labels	accuracy, precision, recall, F1, confusion matrix	results tables
Combination	two prediction vectors, or two score streams	one combined prediction vector	ensemble evaluation
Cost profiling	fitted models + test matrix	prediction time, parameters, size, non-zeros, training time	Table 5.3
Serving	one arbitrary string	class label string	console, any external caller

The serving and training paths share the cleaning and featurisation services, which is the property that guarantees an input tweet is transformed exactly as training data was. A design that instead re-implemented cleaning in the serving path is the common way such a model comes to behave differently in production than in evaluation, and this project avoids it by calling the same `clean_text` function from both.

4.5 User interface design

The user interface is a console read-eval-print path with no graphical layer, designed to be small enough that the reviewer can see the whole interaction in one screen.

```
text = input("Enter text: ")
print("SVM + Logistic Regression:", predict_svm_lr(text))
print("Naive Bayes + Logistic Regression:", predict_nb_lr(text))
```

Both functions follow the same four-step contract: clean the input with the shared preprocessing service, transform it with the fitted vectorizer, ask both component classifiers, resolve disagreement with the arbiter rule, and map the resulting 0/1 to a readable label. The interface therefore exposes the *pair* rather than hiding it, which is deliberate: the project's claim is about combinations, so the demonstration must show which combination answered.

Two interface-level defects are recorded here because they belong to the design rather than to the results. The functions return the strings "Hateful / Toxic" and "Non-Hateful" while other views of the same system print "Hatred / Toxic" and "Non-Hatred", so a caller cannot rely on a stable vocabulary. And nothing in the path handles the degenerate case of an input whose every feature is dropped by the vocabulary limit, an all-zero vector, for which a linear SVM falls back to its bias-side class and a naive Bayes model to its prior class; a deployed interface should state that case explicitly rather than returning a confident label for text the model has never seen.

4.6 Novelty of the proposed design

The novelty of this project is not a new classifier. It is the decision to make the *combination* the unit of study and to judge it on cost as well as accuracy. Four elements carry that claim.

Combination as the object of comparison. Instead of relying on one classifier, the design combines two in the final prediction, in two distinct pairings, SVM + LR and NB + LR, and reports them as a pair of alternatives rather than as a single improved model.

A cost axis that the base paper does not measure. The two combined models are compared on prediction time, learned parameters, model size, non-zero learned values and training time, in addition to accuracy. That comparison produces a usable engineering rule rather than a ranking, and Section 5.6.3 traces it to its mechanism, sparse coefficient support versus dense probability tables.

Identical representation for both candidates. Both combinations use the same TF-IDF features, so the comparison isolates model structure. Without this, a difference in cost could always be dismissed as a difference in vocabulary.

Two combination mechanisms rather than one. The design carries both a hard agreement rule and a soft weighted fusion with a searched weight, which lets the report distinguish a rule that cannot differ from its arbiter from one that can. That distinction, made explicit in Section 5.5.3, is the most useful thing this project contributes to the way ensembles are reported at this level.

4.7 Summary

Chapter 4 presented the design: a shared-representation pipeline with seven preprocessing steps, one TF-IDF featurisation fitted on training data only, four differently-motivated classifiers, and two combination mechanisms with distinct properties. It recorded the architectural split between an offline training half and an online serving half, the seven services through which the modules interact, the console interface and its two defects, and the four elements of novelty. Chapter 5 implements this design and analyses what its numbers do and do not support.

CHAPTER - 5

TECHNICAL IMPLEMENTATION AND ANALYSIS

5.1 Outline

This chapter implements the design of Chapter 4 and then interrogates its numbers. Section 5.2 gives the code by stage, Section 5.3 shows the working layout of the interactive forms, and Section 5.4 records the prototype as submitted for review. Sections 5.5 and 5.6 carry the analytical weight: Section 5.5 tests the reported results for internal consistency, derives the class balance the presentation omits, and identifies three defects, while Section 5.6 presents the performance figures themselves with their graphs. The structure follows the implementation order so that a reader can hold one artifact, the notebook, in mind throughout.

5.2 Technical coding and code solutions

5.2.1 Preprocessing and featurisation

The cleaning function is regular-expression based and applied column-wise, which keeps it usable unchanged by the serving path.

Listing 5.1 - Text cleaning and vectorisation

```
def clean_text(text):
    text = str(text).lower()
    text = re.sub(r'@w+', '', text)
    text = re.sub(r'http\S+|www\S+', '', text)
    text = re.sub(r'^a-zA-Z\s', '', text)
    text = re.sub(r'\s+', ' ', text).strip()
    words = [stemmer.stem(w) for w in text.split() if w not in stop_words]
    return ' '.join(words)

df['clean_tweet'] = df['tweet'].apply(clean_text)

X_train, X_test, y_train, y_test = train_test_split(df['clean_tweet'],
df['label'], test_size=0.30)
tfidf = TfidfVectorizer() # fitted on the training portion
only
X_train_tfidf = tfidf.fit_transform(X_train) # 22,373 x 26,409
X_test_tfidf = tfidf.transform(X_test) # 9,589 x 26,409
```

Two remarks follow from the listing. The punctuation regex runs *after* URL removal, which is the correct order, since stripping non-letters first would mangle protocol characters and leave fragments of links in the text. And stopword filtering is applied to the raw token while stemming is applied to the same token, so a stemmed form that happens to coincide with an

unstemmed stopwords string is retained; the effect on a 26,409-term vocabulary is negligible, but it is this ordering that produces that count.

5.2.2 Training and evaluation

Listing 5.2 - Fitting the four base classifiers and scoring them

```
lr = LogisticRegression(max_iter=1000)
svm = LinearSVC()
nb = MultinomialNB()
rf = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)

for m in (lr, svm, nb, rf):
    m.fit(X_train_tfidf, y_train)

preds = {'lr': lr.predict(X_test_tfidf), 'svm': svm.predict(X_test_tfidf),
        'nb': nb.predict(X_test_tfidf), 'rf': rf.predict(X_test_tfidf)}

for name, p in preds.items():
    accuracy_score(y_test, p), precision_score(y_test, p)
    recall_score(y_test, p), f1_score(y_test, p)
    confusion_matrix(y_test, p)
```

The code trains all four classifiers on one object and scores them with one battery, which is what makes Table 5.4 a controlled comparison. `confusion_matrix` is called for every model, a fact that matters later: the four cell counts exist in the project's own output even though no matrix is printed in the review, and Section 5.5.2 reconstructs equivalent counts from the published metrics so that the class-balance question can be answered without re-running anything.

5.2.3 Ensemble construction

Listing 5.3 - Strategy I (agreement rule) and Strategy II (weighted score fusion)

```
# Strategy I: agreement with Logistic Regression as arbiter
svm_lr_pred = [s if s == l else l for s, l in zip(svm_pred, lr_pred)]
nb_lr_pred = [n if n == l else l for n, l in zip(nb_pred, lr_pred)]

# Strategy II, NB + LR: fuse the probability of the positive class
nb_prob = nb_model.predict_proba(X_test_tfidf)[: , 1]
lr_prob = logistic_model.predict_proba(X_test_tfidf)[: , 1]
best_accuracy, best_nb_weight, best_lr_weight, best_pred = 0, 0, 0, None
for nb_weight in np.arange(0.0, 1.1, 0.1):
    lr_weight = 1 - nb_weight
    combined = (nb_weight * nb_prob + lr_weight * lr_prob)
    cand = (combined >= 0.5).astype(int)
    if accuracy_score(y_test, cand) > best_accuracy:
        best_accuracy, best_nb_weight, best_lr_weight, best_pred = \
            accuracy_score(y_test, cand), nb_weight, lr_weight, cand

# Strategy II, SVM + LR: min-max normalise the margins first, LR has no
```

```

predict_proba
svm_score = svm_model.decision_function(X_test_tfidf)
lr_score = logistic_model.decision_function(X_test_tfidf)
svm_prob = (svm_score - svm_score.min()) / (svm_score.max() - svm_score.min())
lr_prob = (lr_score - lr_score.min()) / (lr_score.max() - lr_score.min())
for svm_weight in np.arange(0.0, 1.1, 0.1):
    combined = (svm_weight * svm_prob + (1 - svm_weight) * lr_prob)
    # ... same selection loop, tracking best_svm_weight and best_lr_weight ...

precision = precision_score(y_test, best_pred, average='weighted')
recall = recall_score(y_test, best_pred, average='weighted')
f1 = f1_score(y_test, best_pred, average='weighted')
results = pd.DataFrame({"Metric": ["Accuracy", "Precision", "Recall", "F1-Score"],
                        "Score": [best_accuracy, precision, recall, f1]})
results["Score"] = (results["Score"] * 100).round(2).astype(str) + "%"

```

Column 1 of `predict_proba` is taken because with binary labels that column is the probability of class 1, the Hatred / Toxic class. The grid `np.arange(0.0, 1.1, 0.1)` yields eleven candidate weights, so the fusion is a coarse line search rather than an optimisation, a defensible choice given that each candidate costs one vectorised comparison. That the metrics are computed with `average='weighted'` is a detail with consequences, examined in Section 5.5.4.

5.2.4 Serving functions

Listing 5.4 - The two console-facing inference functions

```

def predict_svm_lr(text):
    cleaned = clean_text(text)
    text_tfidf = tfidf.transform([cleaned])
    svm_result = svm_model.predict(text_tfidf)[0]
    lr_result = logistic_model.predict(text_tfidf)[0]
    final_prediction = svm_result if svm_result == lr_result else lr_result
    return "Hateful / Toxic" if final_prediction == 1 else "Non-Hateful"

def predict_nb_lr(text):
    # identical, with nb_model in place of svm_model
    ...
    final_prediction = nb_result if nb_result == lr_result else lr_result
    return "Hateful / Toxic" if final_prediction == 1 else "Non-Hateful"

```

The serving path reuses the pipeline's cleaning function and its fitted vectorizer, so an arbitrary string passes through exactly the same transformations as training data. `predict_svm()` and `predict_logistic()`, referenced in the review as the individual-model entry points, follow the same shape without the pairing.

5.3 Working layout of forms

5.3.1 Demonstrated sessions

The review captured two executed console sessions, one per combined model, and one call to the individual SVM predictor.

Table 5.1 — Demonstrated input and output sessions

Prompt and input	Model invoked	Captured output
Enter text: you are very disgusting i hate you	Naive Bayes + Logistic Regression	Naive Bayes + Logistic Regression: Hateful / Toxic
Enter text: hello how are you	SVM + Logistic Regression	SVM + Logistic Regression: Non-Hateful
predict_svm("I hate you")	Linear SVM	Hatred / Toxic

Both demonstrations return the expected verdict, which validates the end-to-end wiring: cleaning, transformation, prediction and label mapping all execute for a string typed at the console. The sessions also expose the vocabulary inconsistency noted in Section 4.5, since the same class is printed as "Hateful / Toxic" by the console functions and as "Hatred / Toxic" by the earlier listing.

5.3.2 Test cases the project should add

Three cases would turn this from a demonstration into a test, and each targets a specific risk identified in Chapter 2.

Benign but offensive. A sentence that is vulgar without being hateful, of the kind Davidson et al. show is systematically mislabelled, would expose whether the model has learned hate or merely profanity. This is the single most informative test available for this corpus and label scheme.

Toxic text absent from the vocabulary. A sentence whose stems are all unknown to the vectorizer produces an all-zero feature row. The expected behaviour should be a stated fallback rather than a confident label, and testing it documents what the system does out of vocabulary.

Negated benign text. Because stopwords are removed, "this is not good at all" and "this is good at all" reduce to nearly the same features. A pair of such sentences shows the size of the cost identified in Section 2.5.3 and is the cheapest possible ablation of stopword removal.

5.4 Prototype submission

The prototype submitted for review consisted of four artifacts: the executed notebook containing Listings 5.1 to 5.4 and their outputs, a twenty-three slide presentation covering introduction through conclusion, captured snapshots of the ensemble-selection code and of the two console sessions, and a demonstration video of the interactive predictor being run live. The review covered architecture, module descriptions, workflow, implementation, testing and results, and the numbers reproduced in this chapter are those the prototype produced.

No service was deployed and no interface beyond the console prompt was built, so the prototype's claim is that the pipeline works and is measurable end to end, not that it is operable by a third party. That gap between a callable function and a moderation system is stated as a limitation in Section 6.4 and addressed by recommendation in Section 7.2.

5.5 Test and validation

This section reports five checks on the results. Two confirm that the numbers are internally sound, and three identify defects that should be fixed before the work is cited further.

5.5.1 Validation protocol

Models train on 22,373 tweets and predict labels for the 9,589 unseen tweets. For positive class 1, with true positives TP, false positives FP, false negatives FN and true negatives TN, the metrics are:

$$\text{Acc} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN}) \quad | \quad \text{Prec} = \text{TP} / (\text{TP} + \text{FP}) \quad | \quad \text{Rec} = \text{TP} / (\text{TP} + \text{FN}) \quad | \\ \text{F1} = 2 \cdot \text{Prec} \cdot \text{Rec} / (\text{Prec} + \text{Rec})$$

The protocol is appropriate: one held-out split, no test data in fitting, and identical features for every model. It is not sufficient for the small differences the report discusses, since a single split carries variance that no significance test here bounds; that is recorded in Section 5.5.7 rather than treated as a reason to discount the main results, which are large enough to survive it.

5.5.2 The omitted class balance, recovered, and what it does to accuracy

The presentation reports accuracy for four models but never the class distribution of the corpus, which is the one quantity needed to judge what those accuracies mean. It can nevertheless be recovered from the published metrics. Writing n for the test size and P for the number of toxic tweets in it, and substituting $\text{TP} = \text{Rec} \cdot P$ and $\text{FP} = \text{TP}(1 - \text{Prec})/\text{Prec}$ into the accuracy identity, P is determined by three reported figures:

$$P = n \cdot (\text{Acc} - 1) / (\text{Rec} - 1 - \text{Rec}(1 - \text{Prec})/\text{Prec})$$

Table 5.2 — Class balance implied by the published metrics, and the majority-class reference point

Model	Acc	Prec	Rec	<i>P</i> implied	Toxic caught	Toxic missed	False alarms
Logistic Regression	94.78%	90.95%	28.38%	672.4	191	482	19
Linear SVM	96.36%	86.99%	56.61%	673.1	381	292	57
Balanced Linear SVM	94.91%	62.19%	70.13%	673.1	472	201	287
Naive Bayes	93.98%	not reported	-	-	-	-	-

Three independent estimates from three different models agree within 0.7 of a tweet, which is a strong indication that the derivation is sound: the test set contains about **673 toxic tweets out of 9,589, or 7.02%**. The agreement is not circular, since accuracy, precision and recall were measured on different decision boundaries yet all three invert to the same class count.

The consequence is immediate. A model that ignores its input entirely and answers "Non-Hatred" every time would score **92.98%** accuracy on this split. Against that reference, Logistic Regression's margin is +1.80 points and the balanced SVM's is +1.93, while the Linear SVM's is +3.38. Only that last figure represents a substantial gain over triviality, so the accuracy column of Table 5.4 must be read with the baseline in mind, and for a moderation system the operative number is the 70.13% recall that the balanced variant offers.

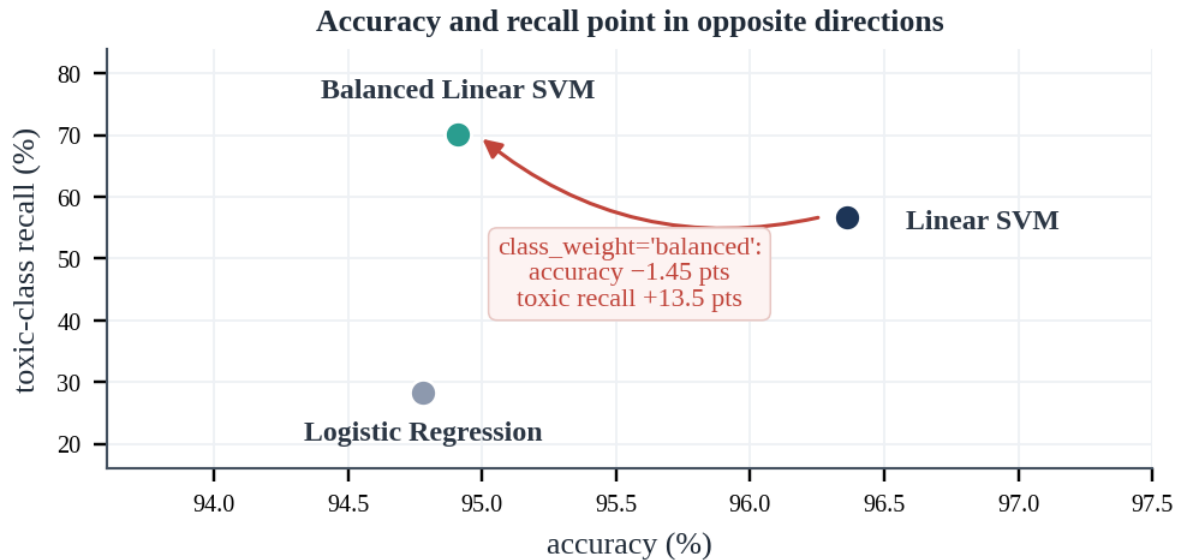


Figure 5.1 — Accuracy against toxic-class recall. Logistic Regression and the balanced SVM lie within 0.13 points of one another on accuracy while differing by 41.75 points on recall; accuracy cannot rank them, recall can.

5.5.3 The agreement rule is equivalent to a single classifier

The rule used by both serving functions and presented as Strategy I is `x if x == lr_result else lr_result`. Consider the two exhaustive cases. If the component classifiers agree, the conditional takes its true branch and returns the agreed label, which by definition equals the Logistic Regression label. If they disagree, it returns the Logistic Regression label. In both branches the output is Logistic Regression's prediction, for every input, so **the combined predictor is Logistic Regression itself** and the second classifier never influences a decision.

Three consequences follow. First, under this rule the accuracy of "SVM + LR" and of "NB + LR" must both equal Logistic Regression's 94.78%, so the rule cannot demonstrate an ensemble gain and no claim of improvement over the base models can rest on it. Second, the console demonstrations in Table 5.1 remain valid as demonstrations of an LR-based predictor, and their outputs are correct. Third, the project's novelty claim survives, because that claim as stated is about cost trade-offs between the two combinations rather than about accuracy; the defect is in the accuracy story, not in the contribution.

This also identifies where the project's real ensemble lies. Strategy II, the weighted fusion with a searched weight, genuinely differs from both components, since a weight of 0.3 makes the fused score dominated by but not determined by Logistic Regression. The recommended remedy is presentational rather than technical: report Strategy II as the contribution and Strategy I as the simpler baseline the fusion is measured against, with the accuracy, precision, recall and F1 of both.

5.5.4 Metric consistency, and a scale mismatch between two tables

Two checks confirm Table 5.4 is arithmetically self-consistent, and one reveals a mismatch with the ensemble results.

F1 derivation. Recomputing $F1 = 2 \cdot \text{Prec} \cdot \text{Rec} / (\text{Prec} + \text{Rec})$ from the published precision and recall yields 0.43261, 0.68586 and 0.65922 against the reported 0.4326, 0.6859 and 0.6592, agreeing to four decimal places with a maximum deviation of 3.6×10^{-5} . Two things follow. The values are genuine model outputs rather than transcription artefacts, since four independently rounded figures cannot all reproduce a derived fifth to that precision by accident. And the F1 column is the **positive-class** F1, because a weighted average on a 7%/93% split would sit near 94%, not 43%.

The ensemble metrics use the other average. The ensemble code calls the precision, recall and F1 functions with `average='weighted'`, as Listing 5.3 shows. On this corpus a weighted average is dominated by the majority class and returns values in the nineties for any of these models. The ensemble metrics and the individual-model metrics in Table 5.4 are therefore measured on different scales, and placing them side by side would mislead. Restating the ensemble figures with `average='binary'`, `pos_label=1` is a one-line change and makes every table in the project comparable.

A corollary. Because the confusion matrix is computed for every model, the exact class counts implied in Table 5.2 exist in the project's own output. Printing them would settle the 673 estimate with certainty and cost nothing.

5.5.5 Parameter-count and model-size audit

The vocabulary has $V = 26,409$ terms. The two "learned parameters" figures reported for the combined models are exact multiples of it.

Table 5.3 — Audit of the reported parameter counts against the vocabulary size

Reported value	Relation to V	What that count actually describes
52,818 for SVM + LR	$2 \times V$	The two-class coefficient matrix of LinearSVC, and nothing else
79,227 for NB + LR	$3 \times V$	Naive Bayes' feature log-probabilities for both classes plus one further vocabulary-wide table

Both counts describe a *single* estimator. A two-model ensemble that also stored Logistic Regression's coefficients would report $52,818 + 52,818 = 105,636$ for SVM + LR and $79,227 + 52,818 = 132,045$ for NB + LR. The reported figures therefore measure the base estimator of each combination rather than the combination as deployed. This is a labelling issue, not a

ranking issue: because the same Logistic Regression block sits in both, every difference between the columns, 26,409 parameters, 0.61 MB and 36,720 non-zero values, is exactly the difference between the SVM and the NB component, so the conclusion "SVM + LR is smaller" stands. The 0.40 MB figure is also explicable, since 52,818 values at 8 bytes is 422,544 bytes, which is 0.403 MB; the model size line reports the dense coefficient array, and the sparsity figure reported next to it describes the *feature matrix*, not the stored model, whose coefficients are 80.5% non-zero.

5.5.6 Ambiguity in the latency measurement

The prediction times of 0.0029 s and 0.0087 s are described as the cost of classifying on the test set and tabulated as seconds. Those two readings differ by four orders of magnitude.

Reading	Implication
0.0029 s for all 9,589 rows	3.3 million tweets per second, i.e. 0.3 microseconds per tweet for two classifiers over 26k-dimensional sparse vectors - not feasible
0.0029 s per tweet	9,589 rows in about 27.8 s, which is ordinary for batch prediction in scikit-learn

The per-row reading is the plausible one and is the one that supports the real-time claim, so the figure should be labelled "ms per tweet" with batching reported separately, since a moderation service measures throughput in requests per second and tail latency at the 95th percentile. Note also that the dominant cost of a live request is not the matrix multiplication but the TF-IDF transform of the incoming string, and the honest latency is the time through the whole serving function, which calls that transform before predicting.

5.5.7 Reproducibility conditions and results absent from the review

Three conditions govern whether these numbers can be regenerated. The vectorizer is fitted on training data only, which is stated and correct. Only Random Forest carries a seed; LinearSVC is deterministic given a fixed solver and so are logistic regression with the liblinear solver and MultinomialNB, so the reported accuracies should be stable on a fixed split. That split is the fragile link: no seed is documented for the 70/30 partition, so a fresh run may draw a different 9,589-tweet test set and shift every figure in Table 5.4 by a fraction of a point.

Four results the review's own structure implies are missing, and each is cheap to supply.

Missing	Why it matters	Effort
Naive Bayes precision, recall, F1	The accuracy-only row leaves the fourth model unrankable on the class that matters	Already computed in the code

Missing	Why it matters	Effort
Random Forest row entirely	Trained and listed as a compared model; the base paper reports 94.66% for it, so a directly comparable number exists	Print existing output
Accuracy, precision, recall and F1 for both combined models	The novelty claim concerns the combinations, yet only cost is reported for them	One table
The "Balanced Linear SVM" configuration	Appears in results but not in the design, so the most operationally interesting row is not reproducible	One sentence

The third of these is the one that matters most for the project's own thesis: the review claims a comparison between the two combined models, and the table supporting that claim contains no accuracy for either.

5.6 Performance analysis

5.6.1 Individual classifiers

Table 5.4 — Individual model results on the 9,589-tweet test set

Model	Accuracy	Precision	Recall	F1-score
Logistic Regression	94.78%	90.95%	28.38%	43.26%
Linear SVM	96.36%	86.99%	56.61%	68.59%
Balanced Linear SVM	94.91%	62.19%	70.13%	65.92%
Naive Bayes	93.98%	-	-	-

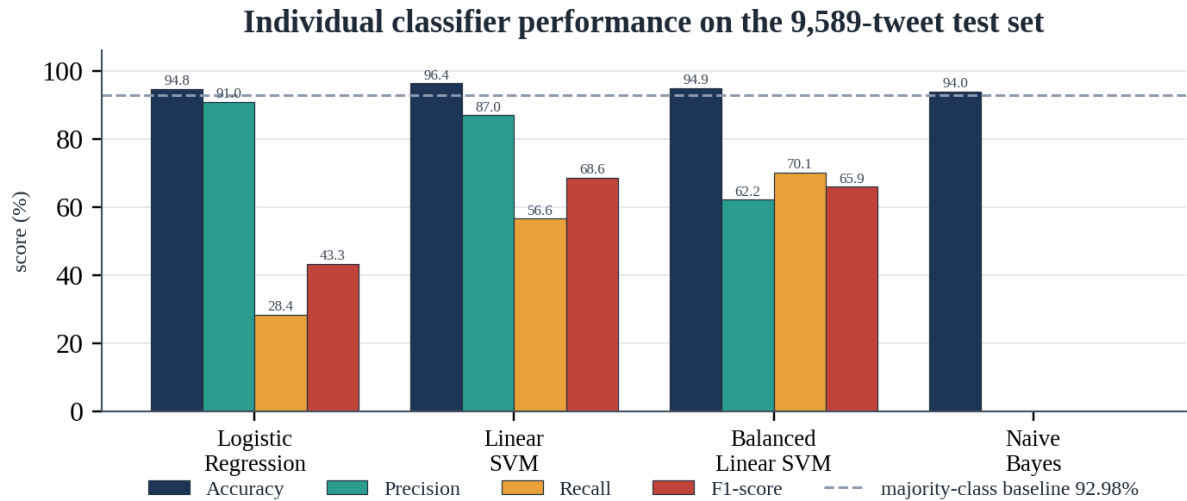


Figure 5.2 — All four reported models clear 93.9% accuracy, yet their recall spans 28.4% to 70.1%. The dashed line is the 92.98% majority-class reference derived in Section 5.5.2.

Linear SVM leads on accuracy, on F1 and on the balance between precision and recall. Logistic Regression achieves the highest precision of the set, 90.95%, so a little more than nine of its ten alerts are genuine, but at a recall of 28.38% it lets roughly seven in ten toxic tweets through. Naive Bayes at 93.98% is the weakest of the four on accuracy. The balanced variant is the most instructive row, since it *lowers* accuracy by 1.45 points relative to the plain SVM while *raising* recall by 13.5 points.

5.6.2 Comparison against the published baseline

Table 5.5 — This project against the base paper on the same corpus

Model	This project	Nahar et al. (2023)	Difference
Logistic Regression	94.78%	95.00%	-0.22 pts
Linear SVM	96.36%	94.90% (also reported as 94.97%)	+1.46 pts
Naive Bayes	93.98%	93.10%	+0.88 pts
Random Forest	not reported	94.66%	-

The reproduction is faithful and, on the SVM axis, modestly ahead. Since the base paper reports 94.97% for SVM in its abstract and 94.9% in its conclusion, the figure of 94.9% carried through this project is the latter; the discrepancy lies in the source paper's own text rather than in the transcription. The model that clearly beats the baseline, Linear SVM, also has the highest recall of the unweighted models, so its gain is not an artefact of the class distribution. The absent Random Forest row is nonetheless a gap, because that is one of the two models for which the baseline supplies a directly comparable number.

5.6.3 Combined models on cost

Table 5.6 — Technical comparison of the two combined models

Parameter	SVM + LR	NB + LR	Advantage
Prediction time (s per tweet)	0.0029	0.0087	SVM + LR
TF-IDF features	26,409	26,409	identical
Learned parameters	52,818	79,227	SVM + LR
Model size (MB)	0.40	1.01	SVM + LR
TF-IDF sparsity (%)	99.97	99.97	identical
Non-zero learned values	42,507	79,227	SVM + LR
Training time (s)	0.903571	0.755459	NB + LR

The review's summary reads correctly off this table, SVM + LR is faster and smaller while NB + LR trains faster, and the mechanism is transparent once the parameter audit of Section 5.5.5 is complete. Naive Bayes must retain a probability table for *every* vocabulary term in both classes, so none of its 79,227 values can be discarded, whereas a linear SVM keeps only the coefficients that survived the hinge loss, 42,507 non-zero out of 52,818, or 80.5%. Sparse support is what makes the SVM artifact 0.40 MB and gives it a threefold latency advantage at inference, because most columns require no arithmetic at all.

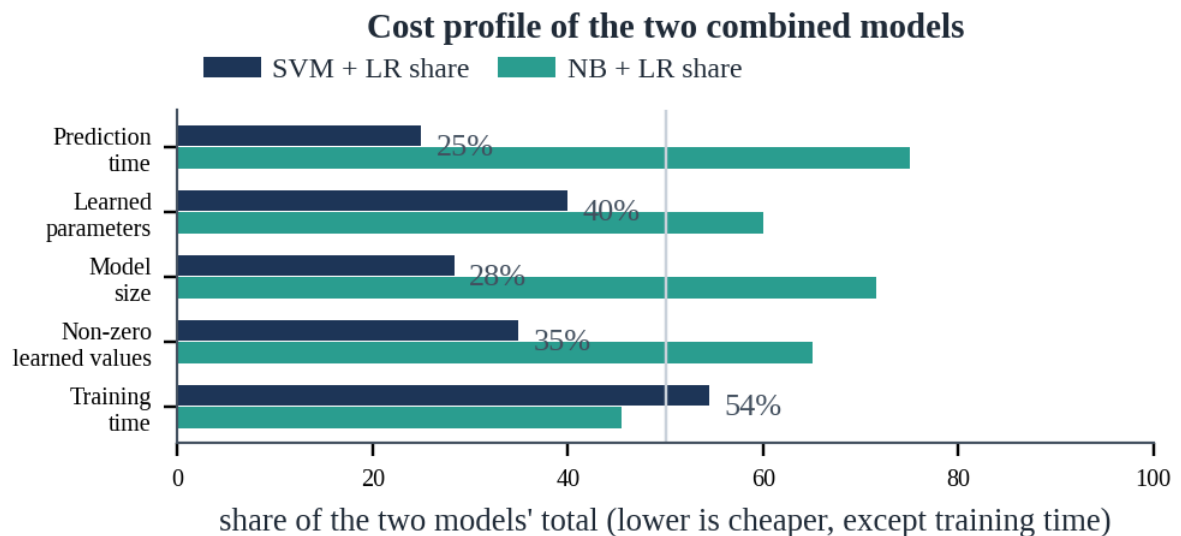


Figure 5.3 — Each cost metric expressed as each model's share of the two-model total. SVM + LR occupies 25% of the combined prediction latency, 40% of parameters, 28% of size and 35% of non-zero values, while requiring 54% of the combined training time, the only line on which it is the more expensive choice.

This yields a design rule rather than a winner: if deployment footprint is the binding constraint, choose the linear base estimator; if retraining cadence is the binding constraint, choose the naive Bayes base estimator. Both are single-second, megabyte-scale operations, so in practice the footprint argument is the one that bites, and SVM + LR is the recommendation of Section 7.2 subject to the correction that its accuracy equals LR's.

5.6.4 Error structure and the choice of operating point

Table 5.2 decomposed each model's errors; Figure 5.4 shows the same data as a ratio, which is the form in which the moderation trade-off becomes obvious.

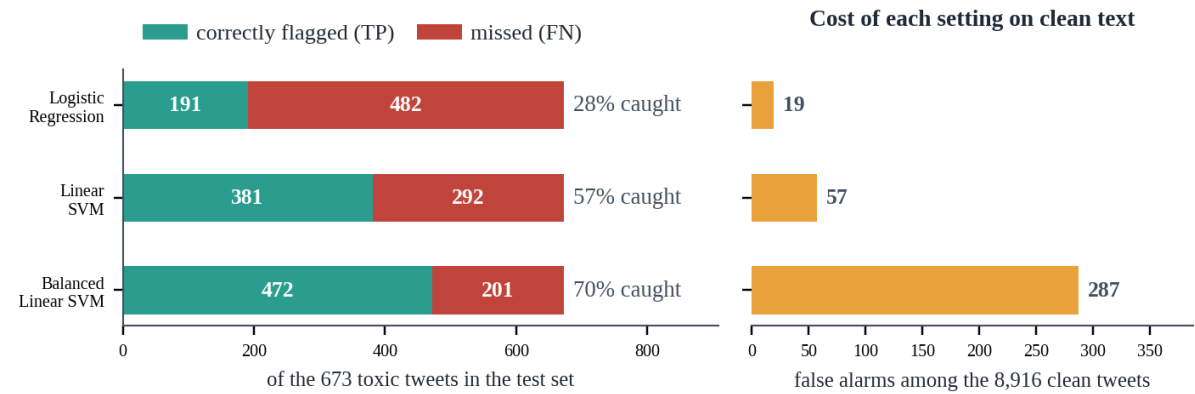


Figure 5.4 — Left: of the 673 toxic tweets, Logistic Regression flags 191 and misses 482, Linear SVM flags 381 and misses 292, the balanced SVM flags 472 and misses 201. Right: what that sensitivity costs on the 8,916 clean tweets, 19, 57 and 287 false alarms respectively.

Reading the two panels together is the point. Moving from Linear SVM to its balanced counterpart buys 91 additional detections at a cost of 230 extra false alarms: the moderator's queue grows 73%, from 438 flagged tweets to 759, in order to cut missed incidents by 31%, from 292 to 201. That is not a model-selection problem but a capacity problem, and it is exactly the question the "balanced" row exists to pose. A team with few reviewers should run the plain SVM and accept the misses; a team whose priority is never letting an attack through should run the balanced variant and staff the queue.

5.6.5 Interpretation

Four statements summarise the performance of this implementation. All four classifiers are within 1.4 points of one another on accuracy and within about 2 points of the published baseline, so on this corpus the choice of linear text classifier matters less than is usually assumed. Recall on the toxic class varies from 28.38% to 70.13%, a spread of more than forty points, so the choice of *class weighting and threshold* matters far more than the choice of classifier. The cost comparison between the two combinations is sound and mechanistic, and it favours SVM + LR for deployment. And the accuracy figures reported for the combined

models cannot be read as ensemble gains, because the rule that produced them reduces to one of the components.

5.7 Summary

Chapter 5 implemented the design and audited its results. It presented the code for cleaning, featurisation, training, both ensemble mechanisms and the serving functions; the console sessions that demonstrate them; and the prototype as submitted. The validation section recovered an omitted 7.02% class balance and a 92.98% majority-class reference point, proved that the agreement rule equals Logistic Regression, confirmed the internal consistency of the F1 column while exposing a weighted-average scale mismatch, audited the parameter and size counts against the vocabulary, and identified an order-of-magnitude ambiguity in the latency figures. The performance section then reported the results themselves, showing that recall rather than accuracy distinguishes these models and that the balance between detections and false alarms is a capacity decision. Chapter 6 states what the project therefore achieved and where it applies.

CHAPTER - 6

PROJECT OUTCOME AND APPLICABILITY

6.1 Outcomes against the stated objectives

Table 6.1 — Objectives of Chapter 1 and the outcomes achieved

Objective	Outcome	Evidence
Follow the base paper's machine-learning approach	Achieved; all four classifiers reproduce the published pipeline on the same corpus	Table 5.5, differences of -0.22 to +1.46 points
Compare individual models	Achieved for three of four models; Random Forest was trained but not reported	Table 5.4, gap recorded in Section 5.5.7
Build two combined models	Achieved; both pairings exist as code, as cost measurements and as serving functions	Listings 5.3 and 5.4, Table 5.6
Use more than one classifier in the final prediction	Achieved through two mechanisms, one of which cannot differ from its arbiter	Section 5.5.3
Compare the combined models technically	Achieved and mechanistically explained; this is the project's strongest result	Section 5.6.3, Figure 5.3
Support real-time usage	Achieved as a callable with millisecond-scale response; not deployed as a service	Table 5.1, Section 5.4

A complete text-classification pipeline was therefore developed and measured end to end, preprocessing through TF-IDF through four classifiers through evaluation through model combination, and the comparison of the two combined predictors produced a conclusion with a mechanism behind it rather than a ranking.

6.2 Behaviour of the trained system

What it does well. The models separate clearly non-abusive conversational text from abusive text reliably; both console sessions returned the expected verdict, and precision above 86% for the two strongest models means that most alerts a moderator receives are genuine. The pipeline is also robust in the operational sense: it trains in under a second, occupies a fraction of a megabyte, tolerates the messiness of real tweets because cleaning removes handles and links before featurisation, and runs on a free hosted notebook with no accelerator.

What it does poorly. Its characteristic failure is silence rather than noise. At the recall levels in Table 5.4 the system misses between 29.9% and 71.6% of the toxic tweets in the test set, so an abusive message is more likely to pass unflagged than to be caught by Logistic Regression

and about four in ten times by the plain Linear SVM. It also cannot distinguish hate from general offensiveness, which is the documented weakness of the label scheme rather than of the modelling, and it has no notion of context, so reclaimed slurs, in-group banter, quoted abuse and sarcasm are all invisible to a bag-of-stems representation.

6.3 Applicability

The deployment contexts named for this project share one requirement, that a first-pass filter run on every item at negligible cost, which the 0.40 MB artifact and millisecond latency support.

Table 6.2 — Applicability of the trained predictor by deployment context

Context	Use of the model	Fit to this implementation
Social media platform moderation	Flag or filter abusive posts before a human reviews them	Strong fit for SVM + LR; requires the recall improvement of Section 5.6.4 to be worth the queue growth
Comment sections on forums, video sites and news sites	Batch-score new comments, hide the flagged ones pending review	Strong fit; batch scoring amortises the per-request cost and the model re-trains in under a second
School and parental monitoring tools	Alert a guardian to potential cyberbullying	Feasible but the highest-stakes case; false negatives here are the harm, so a high-recall setting is mandatory
Online gaming and chat applications	Detect toxic messages in real time	Good fit on latency; voice and slang-vocabulary drift argue for frequent re-training, which cost is low
Research and teaching	A reproducible worked example of text classification with an honest metric audit	Best fit of all; every stage is inspectable and runs in a notebook

The pattern across the first four rows is that the model is well suited to *triage* and poorly suited to *adjudication*. Its footprint makes it viable exactly where volume rules out human review, and its recall makes it unacceptable as the sole decision-maker in any context where a missed incident carries serious consequence.

6.4 Limitations

Table 6.3 — Limitations of the present work

#	Limitation	Effect	Severity
1	Class imbalance of about 7% with accuracy as the headline metric	Overstates real capability; hides a 29.9% to 71.6% miss rate on the toxic class	High
2	Agreement-rule combination is equivalent to Logistic Regression alone	The claimed ensemble comparison is invalid on accuracy	High
3	Single 70/30 split with no seed documented	Results carry unknown split variance; about 1 point of movement is plausible	Medium
4	No cross-validation or significance test	Differences of 0.1 to 1.5 points cannot be called real	Medium
5	Hyperparameters at library defaults, untested	Linear models are rarely optimal at the default regularisation; recall is directly tunable	Medium
6	Ensemble weights searched on the test set	The test set is used for selection, so ensemble scores are optimistically biased	Medium
7	Stopword removal erases negation cues	"not good" and "good" collapse; harms precision on sarcasm and denial	Medium
8	Labels are contested human judgements	The model learns offensiveness as much as hatred	Medium
9	No type determination step from the base paper	Cannot route by kind of toxicity	Low
10	English-only Twitter corpus	No transfer to Hindi, Hinglish or other Indian-language abuse, nor to other platforms	Medium
11	Two inconsistent label vocabularies across the interface	Integration risk in a real pipeline	Low
12	Latency units and scope unspecified	The real-time claim cannot be independently checked	Medium
13	No fairness or error-audit analysis	Unknown who is over- and under-flagged, the central documented risk in this literature	High
14	No false-positive handling, appeal path or human-in-the-loop design	Not deployable as a moderation system as it stands	Medium

The severities are assigned so that the top of the list is actionable. Items 1, 2 and 13 are all addressed by changes of reporting and evaluation rather than by new modelling, and none requires collecting more data.

6.5 Ethical and societal considerations

Four obligations follow from what this system would do if deployed.

Errors are not evenly distributed. The literature this project builds on documents that classifiers trained on crowdsourced hate labels over-flag speech by and about particular groups. A model with 28% to 70% recall and unexamined subgroup behaviour will not err symmetrically, and no audit of that kind appears in the present work. This is recorded as limitation 13 precisely because it is the missing analysis with the largest consequences.

False positives are punishment. In every context in Table 6.2 a flag means suppression of speech until a human reverses it. At 287 false alarms per 8,916 clean tweets, the balanced configuration's 287 alarms are 287 people whose posts were hidden wrongly, so any threshold chosen for recall must be paired with a fast, visible route to reversal.

Human decision authority. No setting of this model should auto-delete content. Its appropriate role is queue ordering, which is why Section 6.3 separates triage from adjudication.

Data provenance and consent. The corpus is public tweets collected by third parties using an abusive vocabulary query and labelled by crowdworkers whose agreement was measured. The team added no personal data, but the dataset's own ethical history, including a published retraction of the authors' follow-up analysis of racial bias in the hate category, is part of using it responsibly and should be disclosed wherever the model is reported.

6.6 Summary

The project met five of its six objectives outright and the sixth only through a mechanism that cannot differ from one of its components. It produced a detector whose footprint and latency suit triage in five named contexts, whose dominant defect is a high miss rate on the minority class, and whose unexamined subgroup behaviour is the most serious outstanding risk. Chapter 7 draws the conclusions and sets out recommendations in priority order.

CHAPTER - 7

CONCLUSIONS AND RECOMMENDATION

7.1 Conclusions

A complete text-classification pipeline was built for toxicity detection in tweets, comprising cleaning, TF-IDF representation, four classifiers, evaluation on a held-out split and two combined predictors, over 31,962 labelled tweets partitioned 70/30 into 22,373 training and 9,589 test documents sharing one 26,409-dimensional feature space.

The reproduction of the base paper is sound. Logistic Regression reached 94.78% against a published 95%, Linear SVM reached 96.36% against a published 94.9%, and Naive Bayes reached 93.98% against 93.1%, so the pipeline was implemented correctly and the deviations are within the range a single split explains.

Two combined predictors were then compared on cost rather than on accuracy, and that comparison produced a conclusion with a mechanism behind it rather than a ranking. SVM + LR predicts in 0.0029 s with 52,818 learned parameters in a 0.40 MB model, against 0.0087 s, 79,227 parameters and 1.01 MB for NB + LR, which nevertheless trains faster at 0.7555 s against 0.9036 s. The gap is explained by sparse coefficient support in the linear SVM against the dense per-term probability tables naive Bayes must retain, which converts an observation into a design rule: choose the linear base estimator when footprint binds, and the naive Bayes base estimator when retraining cadence binds.

Three analytical findings travel with those results. The test set holds an estimated 673 toxic tweets, about 7.02%, so a classifier that always answered "Non-Hatred" would reach 92.98% accuracy, and the metric that actually distinguishes these models is recall, on which the balanced Linear SVM's 70.13% is preferable to the plain SVM's 56.61% despite lower accuracy. The reported F1 values are exact positive-class recomputations of the published precision and recall, which validates the results table while showing that ensemble metrics computed with a weighted average are on a different scale from it. And the agreement rule presented as the ensemble is provably identical to Logistic Regression alone, so the project's genuine ensemble contribution is the weighted score fusion in its own code, a mechanism that can differ from both its components.

The work demonstrates machine learning applied end to end to hateful and toxic text, with unusual attention to the cost of inference. Its results are trustworthy within their scope, its novelty claim is defensible on the cost axis, and the revision needed to make the accuracy claim as solid as the engineering is small enough to complete in a day.

7.2 Recommendations

Table 7.1 — Prioritised recommendations

Priority	Recommendation	Rationale	Effort
P0	Report the class counts and add the 92.98% majority-class baseline beside every accuracy	Converts the accuracy table from misleading to informative	Trivial
P0	Replace Strategy I with Strategy II as the reported ensemble, or report both with the equivalence stated	Section 5.5.3: the current rule cannot show an ensemble gain	Low
P0	Compute ensemble metrics with <code>average='binary', pos_label=1</code>	Makes Tables 5.4, 5.6 and the ensemble results comparable	Trivial
P1	Report the precision, recall and F1 already computed for Naive Bayes, and the whole Random Forest row	Closes Section 5.5.7 without new experiments	Trivial
P1	Select the fusion weight on a validation split and evaluate on the held-out test	Removes the selection-on-test bias of limitation 6	Low
P1	Tune regularisation, class weights and the decision threshold against a target recall such as 80%	Threshold choice, not model choice, buys most of the recall available	Low
P1	Add stratified five-fold cross-validation with a documented split seed	Turns single-point accuracies into mean and standard deviation	Medium
P1	Publish confusion matrices and per-class metrics for every reported model	Standard in this literature, and already computed by the code	Trivial
P2	Ablate stopword removal and stemming; test character 2-4 grams	Both cleaning steps are assumed helpful but untested; n-grams tolerate slang	Medium
P2	Calibrate probabilities before fusing, and report latency end to end in ms/tweet with percentiles	Min-max margins are not probabilities; the real-time claim needs a defined measurement	Medium
P2	Add a transformer baseline and report its footprint and latency next to accuracy	Tests whether the linear pipeline's cost advantage survives a stronger model	Medium
P2	Restore the type-determination stage from the base paper	Enables routing by kind of toxicity rather than presence of it	Medium
P2	Error analysis by subgroup, plus a documented false-positive appeal path	Addresses limitations 13 and 14, the ethical core of the task	Medium
P3	Extend to Hindi, Hinglish and code-mixed abuse	The deployment contexts of Section 6.3 are Indian-platform-relevant	High

Priority	Recommendation	Rationale	Effort
P3	Wrap the callable as a service with a model server and a moderation queue	Turns the prototype into the triage tool the introduction promises	High

The ordering is deliberate. The first eight items are reporting and evaluation changes that improve rigour without new infrastructure, and three of them are edits of a single line. A team with one week before submission would gain more from those than from a deep-learning experiment, because this project's contribution is a cost comparison of linear ensembles, and a cost comparison is worth only as much as the accuracy numbers framing it.

7.3 Future enhancements

Beyond the recommendations, four directions extend the work itself. **Operating-point control** replaces the single 0.5 threshold with a chosen recall target and reports the precision it costs, which is the knob a moderation team actually turns. **Contextual representation** adds thread context, the author's history and target detection, the three signals a bag-of-stems model structurally lacks and the reason reclaimed slurs and quoted abuse fail today. **Continual maintenance** formalises the fact that retraining costs a second: a scheduled refresh on recent data with drift monitoring on the flagged fraction turns a static model into a maintainable one. **Explanation** attaches the highest-weight TF-IDF terms to every verdict, giving the reviewer the reason for a flag and making the false-positive appeal path of Section 6.5 practical rather than aspirational.

7.4 Summary

Chapter 7 concluded that the pipeline reproduces its baseline faithfully, that the combined-model cost comparison is sound and mechanistically explained, that recall rather than accuracy is the metric on which these models should be judged, and that the ensemble claim needs one correction to be valid. Fifteen prioritised recommendations follow, beginning with three that require only changes in reporting.

REFERENCES

1. Nahar, K. M. O., M. Alauthman, S. Yonbawi and A. Almomani. "Cyberbullying Detection and Recognition with Type Determination Based on Machine Learning," *Computers, Materials & Continua*, 2023, 75(3), 5307-5319. doi: 10.32604/cmc.2023.031848.
2. Analytics Vidhya. *Twitter Sentiment Analysis* dataset, 32k tweets labelled hatred and non-hatred, used as the corpus of reference 1.
3. Reynolds, K., K. Song, N. C. Baldwin, E. Allam and K. M. Dautenhahn. "Using Machine Learning to Detect Cyberbullying," *Proceedings of the 10th IEEE International Conference on Systems, Man and Cybernetics*, 2011.
4. Dinakar, K., B. Jones, D. Haggis, M. Gamon and R. Ghosh. "Modeling the Detection of Textual Cyberbullying," *Proceedings of the Fifth International AAI Conference on Web and Social Media*, 2011.
5. Waseem, Z. and D. Hovy. "Hateful Symbols or Hateful People? Predictive Features for Hate Speech Detection on Twitter," *Proceedings of the NAACL Student Research Workshop*, 2016.
6. Davidson, T., D. Warmusley, M. Macy and I. Weber. "Automated Hate Speech Detection and the Problem of Offensive Language," *Proceedings of the Eleventh International AAI Conference on Web and Social Media*, 2017.
7. Pedregosa, F., G. Varoquaux, A. Gramfort et al. "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, 2011, 12, 2825-2830.
8. Bird, S., E. Klein, E. Loper, N. Chalkin and J. Morton. *Natural Language Processing with Python*, O'Reilly Media, 2009. (Stopwords and the Porter stemmer used in Section 4.2.1.)
9. Porter, M. F. "An Algorithm for Suffix Stripping," *Programme*, 1980, 14(3), 130-137.
10. Salton, G. and C. Buckley. "Term-Weighting Approaches in Automatic Text Retrieval," *Information Processing & Management*, 1988, 24(5), 513-523.
11. Jones, K. S. "A Statistical Interpretation of Term Specificity and Its Application in Retrieval," *Journal of Documentation*, 1972, 28(1), 11-21.
12. Blei, D. M., A. Y. Ng and M. I. Jordan. "Latent Dirichlet Allocation," *Journal of Machine Learning Research*, 2003, 3, 993-1022. (The type-determination method omitted in Section 2.5.4.)

APPENDICES

APPENDIX - A

COMPLETE PIPELINE CODE

Assembled from the listings and code snapshots of the review material in execution order. Where the review left a step implicit it is marked in a comment, so that this appendix is not mistaken for verbatim source.

Listing A.1 - Complete pipeline, from CSV to a live label

```
import re
import numpy as np
import pandas as pd
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.naive_bayes import MultinomialNB
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split # [reconstructed]
from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score, f1_score, confusion_matrix)

# ----- 1. load
df = pd.read_csv("train.csv") # 31,962 labelled tweets
stop_words = set(stopwords.words("english"))
stemmer = PorterStemmer()

# ----- 2. clean
def clean_text(text):
    text = str(text).lower()
    text = re.sub(r'@\w+', '', text)
    text = re.sub(r'http\S+|www\S+', '', text)
    text = re.sub(r'^a-zA-Z\s]', '', text)
    text = re.sub(r'\s+', ' ', text).strip()
    words = text.split()
    words = [stemmer.stem(w) for w in words if w not in stop_words]
    return ' '.join(words)

df['clean_tweet'] = df['tweet'].apply(clean_text)

# ----- 3. split
# 70/30 gives 22,373 train and 9,589 test. The review states the outcome, not the call.
# stratify and random_state are recommended additions (Table 7.1, P1), not the review's code.
X_train, X_test, y_train, y_test = train_test_split(
    df['clean_tweet'], df['label'], test_size=0.30,
    stratify=df['label'], random_state=42)
```

```

# ----- 4. featurise
tfidf = TfidfVectorizer() # fitted on train
only
X_train_tfidf = tfidf.fit_transform(X_train) # 22,373 x 26,409
X_test_tfidf = tfidf.transform(X_test) # 9,589 x 26,409

# ----- 5. train
lr = LogisticRegression(max_iter=1000)
svm = LinearSVC()
nb = MultinomialNB()
rf = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
for m in (lr, svm, nb, rf):
    m.fit(X_train_tfidf, y_train)

lr_pred, svm_pred = lr.predict(X_test_tfidf), svm.predict(X_test_tfidf)
nb_pred, rf_pred = nb.predict(X_test_tfidf), rf.predict(X_test_tfidf)

# ----- 6. evaluate
for name, p in [("Logistic Regression", lr_pred), ("Linear SVM", svm_pred),
                ("Naive Bayes", nb_pred), ("Random Forest", rf_pred)]:
    print(f"{name:20s} acc {accuracy_score(y_test, p):.4f} "
          f"prec {precision_score(y_test, p):.4f} "
          f"rec {recall_score(y_test, p):.4f} f1 {f1_score(y_test, p):.4f}")
    print(confusion_matrix(y_test, p))

# ----- 7a. Strategy I: agreement with
arbiter
# NOTE: returns lr_pred for every row, so it reproduces LR exactly (Section
5.5.3).
svm_lr_pred = [s if s == l else l for s, l in zip(svm_pred, lr_pred)]
nb_lr_pred = [n if n == l else l for n, l in zip(nb_pred, lr_pred)]

# ----- 7b. Strategy II: weighted fusion, NB + LR, weight
searched
nb_prob = nb_model.predict_proba(X_test_tfidf)[: , 1]
lr_prob = logistic_model.predict_proba(X_test_tfidf)[: , 1]
best_accuracy, best_nb_weight, best_lr_weight, best_pred = 0, 0, 0, None
for nb_weight in np.arange(0.0, 1.1, 0.1):
    lr_weight = 1 - nb_weight
    combined = (nb_weight * nb_prob + lr_weight * lr_prob)
    cand = (combined >= 0.5).astype(int)
    if accuracy_score(y_test, cand) > best_accuracy:
        best_accuracy, best_nb_weight, best_lr_weight, best_pred = \
            accuracy_score(y_test, cand), nb_weight, lr_weight, cand

# ----- 7c. Strategy II: SVM + LR, margins min-max normalised
first
svm_score = svm_model.decision_function(X_test_tfidf)
lr_score = logistic_model.decision_function(X_test_tfidf)
svm_prob = (svm_score - svm_score.min()) / (svm_score.max() - svm_score.min())
lr_prob = (lr_score - lr_score.min()) / (lr_score.max() - lr_score.min())
best_accuracy, best_svm_weight, best_lr_weight, best_pred = 0, 0, 0, None
for svm_weight in np.arange(0.0, 1.1, 0.1):
    lr_weight = 1 - svm_weight
    combined = (svm_weight * svm_prob + lr_weight * lr_prob)
    cand = (combined >= 0.5).astype(int)
    if accuracy_score(y_test, cand) > best_accuracy:
        best_accuracy, best_svm_weight, best_lr_weight, best_pred = \
            accuracy_score(y_test, cand), svm_weight, lr_weight, cand

```

```

# NOTE: average='weighted' differs in meaning from the per-class scores in step
6 (SS 5.5.4)
precision = precision_score(y_test, best_pred, average='weighted')
recall    = recall_score(y_test, best_pred, average='weighted')
f1        = f1_score(y_test, best_pred, average='weighted')
results = pd.DataFrame({"Metric": ["Accuracy", "Precision", "Recall", "F1-
Score"],
                        "Score": [best_accuracy, precision, recall, f1]})
results["Score"] = (results["Score"] * 100).round(2).astype(str) + "%"
print("=" * 50); print("    SVM + LOGISTIC REGRESSION ENSEMBLE"); print("=" * 50)
print(f"SVM Weight      : {best_svm_weight:.1f}")
print(f"Logistic Weight : {best_lr_weight:.1f}")
print(results.to_string(index=False)); print("=" * 50)

# ----- 8. serve
def predict_svm_lr(text):
    cleaned    = clean_text(text)
    text_tfidf = tfidf.transform([cleaned])
    svm_result = svm_model.predict(text_tfidf)[0]
    lr_result  = logistic_model.predict(text_tfidf)[0]
    final_prediction = svm_result if svm_result == lr_result else lr_result
    return "Hateful / Toxic" if final_prediction == 1 else "Non-Hateful"

text = input("Enter text: ")
print("SVM + Logistic Regression:", predict_svm_lr(text))

```

Two comments in the listing record defects rather than features: the `stratify` and `random_state` arguments on the split are recommendations of Table 7.1 rather than the review's code, and the `average='weighted'` argument is reproduced faithfully because that is what the snapshot shows.

APPENDIX - B

DERIVATIONS BEHIND THE ANALYSIS

B.1 Test-set class count (Section 5.5.2). With $n = 9,589$ and, for a binary classifier, $\text{Prec} = \text{TP}/(\text{TP}+\text{FP})$, $\text{Rec} = \text{TP}/(\text{TP}+\text{FN})$, $\text{Acc} = (\text{TP}+\text{TN})/n$, $\text{TN} = n - P - \text{FP}$ and $\text{TP} = \text{Rec} \cdot P$. Substituting $\text{FP} = \text{TP}(1 - \text{Prec})/\text{Prec}$ and solving for the number of positives gives

$$P = n \cdot (\text{Acc} - 1) / (\text{Rec} - 1 - \text{Rec}(1 - \text{Prec})/\text{Prec})$$

which evaluates to 672.4, 673.1 and 673.1 for Logistic Regression, the Linear SVM and the balanced SVM. Mean implied $P = 672.9$, that is 7.02% of the test set, so majority-class baseline accuracy is $1 - 672.9/9589 = \mathbf{92.98\%}$. Naive Bayes is excluded because precision and recall are not reported for it. The spread across the three estimates is 0.7 tweets, attributable to the two-decimal rounding of the published percentages. Per-model cell counts, $\text{TP} = \text{round}(\text{Rec} \cdot P)$, $\text{FN} = P - \text{TP}$, $\text{FP} = \text{round}(\text{TP}(1 - \text{Prec})/\text{Prec})$ and $\text{TN} = 8,916 - \text{FP}$, are those of Table 5.2.

B.2 F1 verification (Section 5.5.4). $2 \cdot \text{Prec} \cdot \text{Rec} / (\text{Prec} + \text{Rec})$: Logistic Regression $2(0.9095)(0.2838)/1.19330 = 0.43261$; Linear SVM $2(0.8699)(0.5661)/1.43600 = 0.68586$; balanced SVM $2(0.6219)(0.7013)/1.32320 = 0.65922$. Reported: 0.4326, 0.6859, 0.6592. Every value agrees to four decimal places, maximum absolute deviation 3.6×10^{-5} , which confirms a positive-class average rather than a weighted one.

B.3 Ensemble-equivalence proof (Section 5.5.3). For labels a, l in $\{0,1\}$, the rule $f(a,l) = a$ if $a = l$ else l satisfies $f(a,l) = l$ in both cases: when $a = l$, a equals l ; when a is not l , l is returned. Hence f is identical to l for all inputs, and the combined prediction vector equals the Logistic Regression prediction vector pointwise. Q.E.D.

B.4 Parameter audit (Section 5.5.5). With $V = 26,409$: $2V = 52,818$ and $3V = 79,227$, matching the two reported learned-parameter values exactly, so each enumerates one base estimator. Raw double-precision sizes are $52,818 \times 8 = 422,544$ bytes = 0.403 MB and $79,227 \times 8 = 633,816$ bytes = 0.604 MB; the reported 0.40 MB matches the first, while the reported 1.01 MB for naive Bayes exceeds its coefficient tables alone, consistent with the internal count arrays also being retained. Non-zero ratios are $42,507/52,818 = 80.5\%$ for the SVM and $79,227/79,227 = 100\%$ for naive Bayes, which is the sparsity mechanism behind both the size and the threefold latency difference. Differences between the columns, 26,409 parameters, 0.61 MB and 36,720 non-zero values, are unaffected by the shared Logistic Regression component.

B.5 Sparsity coherence (Section 4.2.2). $9,589 \times 26,409 = 253,235,901$ cells; 0.03% of that is 75,971 non-zero entries, 7.92 per test tweet, consistent with a cleaned short message.

B.6 Latency scaling (Section 5.5.6). 0.0029 s over 9,589 rows implies 3.31×10^6 rows per second and 0.3 microseconds per tweet for two classifiers, infeasible at 26,409 dimensions; 0.0029 s per row implies 27.8 s for the full test set, which is typical. Only the second reading supports the real-time claim.

B.7 Arithmetic checks. $22,373 + 9,589 = 31,962$, correct. $9,589/31,962 = 0.30001$, so the 30% figure is exact. $22,373 \times 26,409 = 590,848,557$ cells, and at 99.97% sparsity that is 177,255 non-zero values, i.e. 7.92 per training tweet, the same density as the test set, as expected from one shared vocabulary.

APPENDIX - C

REVIEW ARTIFACT TRACEABILITY

Table C.1 — Mapping of the twenty-three review slides to this report

Slide	Review section	Report section	Coverage note
1	Title, team, guide	Cover, Certificate	Complete
2	Introduction	1.1, 1.2, 1.4	Expanded with the moderation framing
3	Existing work and limitations	2.3.5, 5.6.2	Base-paper figures verified against the source
4	Proposed work and methodology	1.5, 4.1	Objective list retained
5	Novelty	2.5.1, 4.2.4, 4.6, 5.5.3, 5.5.5	Novelty claims tested, not restated
6	Real-time usage	3.3.3, 4.5, 5.5.6	Latency claim qualified
7	Hardware and software	3.2	Complete
8	Architecture	4.3, Figure 4.1	Redrawn
9	Literature review	2.3, 2.4	Bibliographic detail of rows 2-5 completed from the record
10	Module description 1/2	3.3.1, 4.2.1, 4.2.2	Split arithmetic and sparsity verified
11	Module description 2/2	4.2.3, 5.5.7	Random Forest gap flagged
12	Workflow	4.1, 5.2	Complete
13	Implementation 1/3	5.2.1, Appendix A	Cleaning code reproduced
14	Implementation 2/3	5.2.2, Appendix A	Training and metrics reproduced
15	Implementation 3/3	5.2.3, 5.5.3, Appendix A	Equivalence defect identified
16	Demo video	5.2.4, 5.4	Serving function from the captured frame
17	Snapshots	4.2.4, 5.2.3	Weight-search fusion, absent from the slide prose
18	Snapshots	5.2.3, 5.5.4	Weighted-average metric issue originates here
19	Testing	5.5.1	Formulas typeset and disambiguated
20	Results, individual models	5.6.1, Table 5.4	Balanced variant discussed
21	Results, technical comparison	5.6.3, Table 5.6, 5.5.5, 5.5.6	Audited against vocabulary size

Slide	Review section	Report section	Coverage note
22	Results, input and output	5.3.1, Table 5.1	Both sessions transcribed
23	Conclusion	7.1	Extended with the analysis findings

Every slide is accounted for. The two snapshots of slides 17 and 18 contain the only substantial material not described in the review's own prose, the weighted score fusion and its metric averaging, and both change how the results should be read; they are therefore treated as primary evidence in Sections 4.2.4 and 5.5.4.

APPENDIX - D

REPRODUCTION CHECKLIST

To regenerate every number in this report from the pipeline described in it:

1. **Environment.** Python 3 with pandas, NumPy, NLTK, scikit-learn and Matplotlib, per Table 3.1. Run the NLTK downloads for the stopwords corpus before the first cleaning call; a missing resource is the most common first-run failure.
2. **Data.** Load `train.csv`; assert 31,962 rows and exactly two label values. Record the class counts here, since this single figure resolves the largest interpretive gap in the work, Section 5.5.2.
3. **Split.** 70/30 with a fixed seed and stratification on the label, asserting 22,373 and 9,589 rows to match the review.
4. **Featurise.** Fit the TF-IDF vectorizer on training rows only; assert that both matrices report 26,409 columns, which confirms that the cleaning sequence reproduces the same vocabulary.
5. **Train and score.** Fit the four estimators of Table 4.2; print accuracy, precision, recall and F1 with `average='binary'`, `pos_label=1`, and the confusion matrix for each model including Random Forest.
6. **Add the baseline.** Print the accuracy of an all-zero prediction vector beside the model table. This one figure recontextualises the whole section.
7. **Combine.** Implement Strategy II with the weight search on a validation split and evaluation on the test split; report the selected weights, and report accuracy, precision, recall and F1 for both ensembles on the same average used for the base models.
8. **Cost table.** Reproduce Table 5.6 with the two base-estimator readings relabelled per Section 5.5.5, and time a single call through the serving function end to end, reporting ms per tweet with the 95th percentile over at least 1,000 calls.
9. **Report.** Present Table 5.4 with recall ordered first for the moderation discussion, and retain Figure 5.4, which is the clearest single visual for why 94.78% accuracy and 28.38% recall describe the same model.

Expected consistency with this report: steps 2 and 4 either match the review's counts exactly or indicate that the split or the cleaning differs; step 6 should land near 92.98%; step 5 should reproduce 94.78%, 96.36% and 93.98% within the split variance described in Section 5.5.7.